analyze-activity.py

```
import json
import time
import requests
import urllib.request
import urllib.parse
import re
from collections import defaultdict
from market_model import (
    build_recommendations,
    save_recommendations_json,
    filter_recent_signal_metrics_rows,
    MODEL_HISTORY_LOOKBACK_HOURS,
)

PUSHOVER_ENABLED = True
PUSHOVER_USER_KEYS = [
    "u2pfi3ci1na4uujiwekmruofti7nnd",
    "udfwfqtyubkq1tnvdqx84o6h2i78gy"
]
PUSHOVER_API_TOKEN = "agxdaaoicjeeba5pku3znj4i5eahtz"
PUSHOVER_PRIORITY = 0
PUSHOVER_TEST_ALERT = False

HOURS_LOOKBACK = 12
POLL_SECONDS = 60

LEADERBOARD_WALLET_LIMIT = 50
LEADERBOARD_WALLET_OFFSETS = [0 , 50 , 100]

TRACKED_WALLETS = []
WALLET_WEIGHTS = {}

ACTIVE_WALLET_FAILURE_COUNTS = {}
ACTIVE_WALLET_MAX_FAILURES = 3

CONSENSUS_UPGRADE_MIN_CLV = 1.0
CONSENSUS_UPGRADE_MAX_AGE_SECONDS = 300
CONSENSUS_UPGRADE_MIN_WEIGHTED_SCORE = 2.0
PAIRED_RECENT_WINDOW_SECONDS = 20 * 60
BET_ALERT_COOLDOWN_SECONDS = 15 * 60
BET_ALERT_MIN_PRICE_IMPROVEMENT = 0.01
BET_ALERT_MAX_ADVERSE_PRICE_MOVE = 0.04
BET_ALERT_MIN_STAKE_PCT_INCREASE = 10
import os
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

if os.path.isdir("/data"):
    DATA_DIR = "/data"
else:
    DATA_DIR = os.path.join(BASE_DIR, "data")

os.makedirs(DATA_DIR, exist_ok=True)

CLV_TRACKER_PATH = f"{DATA_DIR}/clv_tracker.json"
TRACKED_BETS_PATH = f"{DATA_DIR}/tracked_bets.json"
ALERTED_BETS_PATH = f"{DATA_DIR}/alerted_bets.json"
SIGNAL_METRICS_HISTORY_PATH = f"{DATA_DIR}/signal_metrics_history.json"
SNAPSHOT_CLV_MIN_AGE_SECONDS = 300
SNAPSHOT_CLV_MAX_AGE_SECONDS = 6 * 60 * 60
BET_ALERT_MIN_NEW_SHARP_STAKE = 1000
BET_ALERT_MIN_NEW_BUYS = 2
BET_ALERT_MIN_FOLLOWER_INCREASE = 1
BET_ALERT_MIN_SCORE_IMPROVEMENT = 5
BET_ALERT_MIN_CONSENSUS_SCORE_IMPROVEMENT = 20
BET_ALERT_SOFT_MIN_TOTAL_NOTIONAL = 1000
BET_ALERT_SOFT_MIN_SIZE_RATIO = 1.5
BET_ALERT_SOFT_MIN_LEADERBOARD_ROI = 0.02
BET_ALERT_SOFT_STRONG_SCORE = 85
BET_ALERT_SOFT_STRONG_TOTAL_NOTIONAL = 5000
BET_ALERT_SOFT_STRONG_SIZE_RATIO = 2.5
BET_ALERT_SOFT_STRONG_LEADERBOARD_ROI = 0.05
BET_ALERT_SOFT_STRONG_FOLLOWERS = 1
BET_ALERT_SOFT_STRONG_CONSENSUS_SCORE = 60
BET_ALERT_SOFT_STRONG_MAX_MINUTES_TO_START = 30
BET_ALERT_SOFT_REQUIRED_JUSTIFICATIONS_ONE_FAIL = 2
BET_ALERT_SOFT_REQUIRED_JUSTIFICATIONS_MULTI_FAIL = 3


def fetch_json_url(url):
    req = urllib.request.Request(
        url,
        headers={
            "User-Agent": "Mozilla/5.0",
            "Accept": "application/json",
        },
    )
```

```
    with urllib.request.urlopen(req, timeout=20) as response:
        raw = response.read().decode("utf-8")
        return json.loads(raw)


def load_leaderboard_wallets(limit=25, offsets=None):
    if offsets is None:
        offsets = [0]

    wallets = []
    seen = set()
    raw_rows = []

    safe_limit = max(1, min(int(limit), 50))

    for offset in offsets:
        params = {
            "category": "SPORTS",
            "timePeriod": "ALL",
            "orderBy": "PNL",
            "limit": safe_limit,
            "offset": int(offset),
        }

        query = urllib.parse.urlencode(params)
        url = f"https://data-api.polymarket.com/v1/leaderboard?{query}"

        try:
            data = fetch_json_url(url)
        except Exception as e:
            print(f"[Leaderboard fetch error] offset={offset} -> {repr(e)}")
            continue

        if not isinstance(data, list):
            print(f"[Leaderboard parse warning] Expected list, got {type(data)}")
            continue

        for row in data:
            if not isinstance(row, dict):
                continue

            raw_rows.append(row)

            wallet = str(row.get("proxyWallet", "") or "").strip().lower()
```

```
        if not wallet.startswith("0x"):
            continue
        if len(wallet) != 42:
            continue

        if wallet not in seen:
            seen.add(wallet)
            wallets.append(wallet)

    return wallets, raw_rows

def build_leaderboard_roi_map(leaderboard_rows):
    roi_map = {}

    for row in leaderboard_rows:
        if not isinstance(row, dict):
            continue

        wallet = str(row.get("proxyWallet", "") or "").strip().lower()
        if not wallet.startswith("0x"):
            continue
        if len(wallet) != 42:
            continue

        try:
            pnl = float(row.get("pnl", 0) or 0)
        except (TypeError, ValueError):
            pnl = 0.0

        try:
            vol = float(row.get("vol", 0) or 0)
        except (TypeError, ValueError):
            vol = 0.0

        leaderboard_roi = None
        if vol > 0:
            leaderboard_roi = pnl / vol

        roi_map[wallet] = {
            "leaderboard_pnl": pnl,
            "leaderboard_vol": vol,
            "leaderboard_roi": leaderboard_roi,
            "leaderboard_rank": row.get("rank"),
```

```
        "leaderboard_username": row.get("userName", ""),
    }

    return roi_map


def enrich_wallet_profiles_with_leaderboard(wallet_profiles, leaderboard_rows):
    roi_map = build_leaderboard_roi_map(leaderboard_rows)

    for wallet, stats in roi_map.items():
        if wallet not in wallet_profiles:
            continue

        wallet_profiles[wallet]["leaderboard_pnl"] = stats.get("leaderboard_pnl")
        wallet_profiles[wallet]["leaderboard_vol"] = stats.get("leaderboard_vol")
        wallet_profiles[wallet]["leaderboard_roi"] = stats.get("leaderboard_roi")
        wallet_profiles[wallet]["leaderboard_rank"] = stats.get("leaderboard_rank")
        wallet_profiles[wallet]["leaderboard_username"] = stats.get("leaderboard_username", "")

def load_activity(user_wallet: str):
    encoded_user = urllib.parse.quote(user_wallet, safe="")
    url = f"https://data-api.polymarket.com/activity?user={encoded_user}"

    data = fetch_json_url(url)

    if not isinstance(data, list):
        raise ValueError("Expected activity API response to be a list.")

    return [x for x in data if isinstance(x, dict)]


def filter_recent_trades(trades, hours_lookback=12):
    now_ts = int(time.time())
    cutoff_ts = now_ts - (hours_lookback * 60 * 60)

    recent = []
    for t in trades:
        if not isinstance(t, dict):
            continue
        ts = int(t.get("timestamp", 0) or 0)
        if ts >= cutoff_ts:
            recent.append(t)

    return recent, cutoff_ts, now_ts
```

```
def filter_valid_buy_trades(trades):
    valid = []
    for t in trades:
        if not isinstance(t, dict):
            continue

        side = str(t.get("side", "") or "").upper()
        outcome = str(t.get("outcome", "") or "").strip()
        price = float(t.get("price", 0) or 0)
        size = float(t.get("size", 0) or 0)

        if side != "BUY":
            continue
        if not outcome:
            continue
        if price <= 0:
            continue
        if size <= 0:
            continue

        valid.append(t)

    return valid


def compute_wallet_medians(trades):
    wallet_notionals = defaultdict(list)

    for t in trades:
        if not isinstance(t, dict):
            continue

        wallet = t.get("proxyWallet", "")
        size = float(t.get("size", 0) or 0)
        price = float(t.get("price", 0) or 0)
        notional = size * price

        if notional >= 50:
            wallet_notionals[wallet].append(notional)

    wallet_medians = {}
```

```
    for wallet, notionals in wallet_notionals.items():
        notionals = sorted(notionals)

        if len(notionals) == 0:
            wallet_medians[wallet] = 0
            continue

        if len(notionals) < 5:
            wallet_medians[wallet] = sum(notionals) / len(notionals)
            continue

        n = len(notionals)
        if n % 2 == 1:
            wallet_medians[wallet] = notionals[n // 2]
        else:
            wallet_medians[wallet] = (notionals[n // 2 - 1] + notionals[n // 2]) / 2

    return wallet_medians


def group_accumulation_candidates(trades):
    grouped = defaultdict(list)
    for t in trades:
        if not isinstance(t, dict):
            continue
        wallet = t.get("proxyWallet", "")

        username = (
            t.get("username")
            or t.get("handle")
            or t.get("profileName")
            or ""
        )

        slug = t.get("slug", "")
        outcome = t.get("outcome", "")

        grouped[(wallet, slug, outcome)].append({
            **t,
            "username": username
        })
    results = []

    for (wallet, slug, outcome), group in grouped.items():
```

```
        group = sorted(group, key=lambda x: int(x.get("timestamp", 0) or 0), reverse=True)
        buy_count = len(group)
        total_size = round(sum(float(t.get("size", 0) or 0) for t in group), 6)
        total_weighted_price = sum(
            float(t.get("price", 0) or 0) * float(t.get("size", 0) or 0) for t in group
        )
        total_size_raw = sum(float(t.get("size", 0) or 0) for t in group)
        avg_trade_price = round(total_weighted_price / max(total_size_raw, 1e-9), 6)
        title = group[0].get("title", "")
        first_ts = int(group[-1].get("timestamp", 0) or 0)
        last_ts = int(group[0].get("timestamp", 0) or 0)
        seconds_span = last_ts - first_ts

        if buy_count == 1:
            accumulation_points = 0
        elif buy_count == 2:
            accumulation_points = 10
        elif 3 <= buy_count <= 4:
            accumulation_points = 20
        else:
            accumulation_points = 30

        results.append(
            {
                "wallet": wallet,
                "username": group[0].get("username", ""),
                "title": title,
                "slug": slug,
                "outcome": outcome,
                "buy_count": buy_count,
                "total_size": total_size,
                "avg_trade_price": avg_trade_price,
                "first_timestamp": first_ts,
                "last_timestamp": last_ts,
                "seconds_span": seconds_span,
                "accumulation_points": accumulation_points,
            }
        )

    results = sorted(results, key=lambda r: (r["buy_count"], r["total_size"]), reverse=True)
    return results


def build_fair_price_lookup(accumulation_groups):
```

```
    fair_price_lookup = {}
    grouped = defaultdict(list)

    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue

        slug = str(g.get("slug", "") or "")
        outcome = str(g.get("outcome", "") or "")
        if not slug or not outcome:
            continue

        grouped[(slug, outcome)].append(g)

    for key, rows in grouped.items():
        total_weight = 0.0
        weighted_price_sum = 0.0

        for row in rows:
            try:
                price = float(row.get("avg_trade_price", 0) or 0)
                size = float(row.get("total_size", 0) or 0)
            except Exception:
                continue

            if price <= 0 or size <= 0:
                continue

            weighted_price_sum += price * size
            total_weight += size

        if total_weight > 0:
            fair_price_lookup[key] = round(weighted_price_sum / total_weight, 6)

    return fair_price_lookup


def build_fair_price_lookup(accumulation_groups):
    fair_price_lookup = {}
    grouped = defaultdict(list)

    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue
```

```
        slug = str(g.get("slug", "") or "")
        outcome = str(g.get("outcome", "") or "")
        if not slug or not outcome:
            continue

        grouped[(slug, outcome)].append(g)

    for key, rows in grouped.items():
        total_weight = 0.0
        weighted_price_sum = 0.0

        for row in rows:
            try:
                price = float(row.get("avg_trade_price", 0) or 0)
                size = float(row.get("total_size", 0) or 0)
            except Exception:
                continue

            if price <= 0 or size <= 0:
                continue

            weighted_price_sum += price * size
            total_weight += size

        if total_weight > 0:
            fair_price_lookup[key] = round(weighted_price_sum / total_weight, 6)

    return fair_price_lookup

def build_fair_price_lookup(accumulation_groups):
    fair_price_lookup = {}

    grouped = defaultdict(list)
    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue

        slug = str(g.get("slug", "") or "")
        outcome = str(g.get("outcome", "") or "")
        if not slug or not outcome:
            continue

        grouped[(slug, outcome)].append(g)
```

```
    for key, rows in grouped.items():
        total_weight = 0.0
        weighted_price_sum = 0.0

        for row in rows:
            try:
                avg_trade_price = float(row.get("avg_trade_price", 0) or 0)
            except Exception:
                avg_trade_price = 0.0

            try:
                total_size = float(row.get("total_size", 0) or 0)
            except Exception:
                total_size = 0.0

            if avg_trade_price <= 0 or total_size <= 0:
                continue

            weighted_price_sum += avg_trade_price * total_size
            total_weight += total_size

        if total_weight > 0:
            fair_price_lookup[key] = round(weighted_price_sum / total_weight, 6)

    return fair_price_lookup


def mark_recent_paired_activity(groups):
    ABSOLUTE_OPPOSITE_SIZE_THRESHOLD = 1000
    RELATIVE_OPPOSITE_SIZE_THRESHOLD = 0.35
    STRONG_OPPOSITE_SIZE_THRESHOLD = 2500
    VERY_STRONG_RELATIVE_OPPOSITE_THRESHOLD = 0.60

    grouped_by_wallet_slug = defaultdict(list)

    for g in groups:
        if not isinstance(g, dict):
            continue
        key = (g["wallet"], g["slug"])
        grouped_by_wallet_slug[key].append(g)

    clean_groups = []
```

```
    for g in groups:
        if not isinstance(g, dict):
            continue

        key = (g["wallet"], g["slug"])
        this_outcome = g["outcome"]
        this_last_ts = int(g.get("last_timestamp", 0) or 0)
        this_total_size = float(g.get("total_size", 0) or 0)

        paired_recent = False
        paired_recent_reason = None

        for other in grouped_by_wallet_slug[key]:
            if not isinstance(other, dict):
                continue

            other_outcome = other.get("outcome")
            other_last_ts = int(other.get("last_timestamp", 0) or 0)
            other_total_size = float(other.get("total_size", 0) or 0)

            if other_outcome == this_outcome:
                continue

            time_gap_seconds = abs(this_last_ts - other_last_ts)
            is_recent = time_gap_seconds <= PAIRED_RECENT_WINDOW_SECONDS

            if not is_recent:
                continue

            relative_opposite_size = 0.0
            if this_total_size > 0:
                relative_opposite_size = other_total_size / this_total_size

            strong_absolute_opposite = other_total_size >=
STRONG_OPPOSITE_SIZE_THRESHOLD
            strong_relative_opposite = relative_opposite_size >=
VERY_STRONG_RELATIVE_OPPOSITE_THRESHOLD
            meaningful_opposite = (
                other_total_size >= ABSOLUTE_OPPOSITE_SIZE_THRESHOLD
                and relative_opposite_size >= RELATIVE_OPPOSITE_SIZE_THRESHOLD
            )

            if strong_absolute_opposite or strong_relative_opposite or meaningful_opposite:
                paired_recent = True
```

```
            paired_recent_reason = (
                f"Opposite-side activity within {time_gap_seconds}s "
                f"(opp_size={round(other_total_size, 2)}, "
                f"this_size={round(this_total_size, 2)}, "
                f"opp_ratio={round(relative_opposite_size, 3)})"
            )
            break

        g["paired_recent"] = paired_recent
        g["paired_recent_reason"] = paired_recent_reason
        clean_groups.append(g)

    return clean_groups


def classify_group_role(group):
    if not isinstance(group, dict):
        return "follower"

    buy_count = int(group.get("buy_count", 0) or 0)
    seconds_span = int(group.get("seconds_span", 0) or 0)

    if buy_count <= 1:
        return "leader"

    intervals = max(buy_count - 1, 1)
    avg_seconds_per_buy = seconds_span / intervals

    if buy_count == 2 and seconds_span <= 180:
        return "leader"

    if avg_seconds_per_buy <= 45:
        return "leader"

    if avg_seconds_per_buy <= 180:
        return "early"

    return "follower"


def apply_cross_wallet_sequence_roles(groups):
    grouped = defaultdict(list)

    for g in groups:
```

```
        if not isinstance(g, dict):
            continue
        key = (g.get("slug", ""), g.get("outcome", ""))
        grouped[key].append(g)

    updated_groups = []

    for _, bucket in grouped.items():
        wallet_entries = []

        for g in bucket:
            first_ts = int(g.get("first_timestamp", 0) or 0)
            wallet_entries.append((first_ts, g))

        wallet_entries.sort(key=lambda x: x[0])

        if len(wallet_entries) == 1:
            only_group = dict(wallet_entries[0][1])
            only_group["sequence_role"] = classify_group_role(only_group)
            updated_groups.append(only_group)
            continue

        leader_ts = wallet_entries[0][0]

        for idx, (first_ts, g) in enumerate(wallet_entries):
            g = dict(g)

            if idx == 0:
                g["sequence_role"] = "leader"
            elif idx == 1 and (first_ts - leader_ts) <= 300:
                g["sequence_role"] = "early"
            else:
                g["sequence_role"] = "follower"

            updated_groups.append(g)

    return updated_groups


def is_actionable_accumulation_group(group):
    if not isinstance(group, dict):
        return False
    avg_price = float(group.get("avg_trade_price", 0) or 0)
    return 0.05 <= avg_price <= 0.95
```

```
def load_positions(user_wallet: str):
    encoded_user = urllib.parse.quote(user_wallet, safe="")
    url = f"https://data-api.polymarket.com/positions?user={encoded_user}"

    data = fetch_json_url(url)

    if not isinstance(data, list):
        raise ValueError("Expected positions API response to be a list.")

    return [x for x in data if isinstance(x, dict)]


def build_position_lookup(positions):
    lookup = {}

    for pos in positions:
        if not isinstance(pos, dict):
            continue

        key = (
            pos.get("proxyWallet", ""),
            pos.get("slug", ""),
            pos.get("outcome", ""),
        )
        lookup[key] = pos

    return lookup


def init_wallet_profiles(tracked_wallets):
    profiles = {}

    for wallet in tracked_wallets:
        profiles[wallet] = {
            "wallet": wallet,
            "evaluated_trades": 0,
            "evaluated_clusters": 0,
            "clv_observations": 0,
            "positive_clv_count": 0,
            "avg_forward_clv": 0.0,
            "positive_clv_rate": 0.0,
            "leader_count": 0,
```

```
            "early_count": 0,
            "follower_count": 0,
            "paired_count": 0,
            "noise_count": 0,
            "confidence": 0.0,
            "dynamic_weight": 1.0,
        }

    return profiles


def update_wallet_profiles(wallet_profiles, accumulation_groups, scored_candidates):
    scored_index = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        scored_index[(g.get("wallet", ""), g.get("slug", ""), g.get("outcome", ""))] = g

    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue

        wallet = g.get("wallet", "")
        if wallet not in wallet_profiles:
            wallet_profiles[wallet] = {
                "wallet": wallet,
                "evaluated_trades": 0,
                "evaluated_clusters": 0,
                "clv_observations": 0,
                "positive_clv_count": 0,
                "avg_forward_clv": 0.0,
                "positive_clv_rate": 0.0,
                "leader_count": 0,
                "early_count": 0,
                "follower_count": 0,
                "paired_count": 0,
                "noise_count": 0,
                "confidence": 0.0,
                "dynamic_weight": 1.0,
            }

        profile = wallet_profiles[wallet]
        profile["evaluated_clusters"] += 1
```

```
role = g.get("sequence_role") or classify_group_role(g)
if role == "leader":
    profile["leader_count"] += 1
elif role == "early":
    profile["early_count"] += 1
else:
    profile["follower_count"] += 1

if g.get("paired_recent"):
    profile["paired_count"] += 1

scored_match = scored_index.get((wallet, g.get("slug", ""), g.get("outcome", "")))
if scored_match:
    profile["evaluated_trades"] += 1

    market_movement_cents = scored_match.get("market_movement_cents")
    age = int(scored_match.get("seconds_since_last_buy", 999999) or 999999)

    if market_movement_cents is not None and age <= 1800:
        market_movement_cents = float(market_movement_cents)
        old_n = profile["clv_observations"]
        new_n = old_n + 1

        profile["avg_forward_clv"] = round(
            ((profile["avg_forward_clv"] * old_n) + market_movement_cents) / new_n,
            4
        )
        profile["clv_observations"] = new_n

        if market_movement_cents > 0:
            profile["positive_clv_count"] += 1

        profile["positive_clv_rate"] = round(
            profile["positive_clv_count"] / max(profile["clv_observations"], 1),
            4
        )

    reason = str(scored_match.get("reason", "") or "").lower()

    if (
        "soft-filter" in reason
        or "too late" in reason
        or "stale" in reason
        or "no positive clv" in reason
```

```
        ):
            profile["noise_count"] += 1

    return wallet_profiles


def filter_active_wallets(wallet_profiles):
    filtered = []

    for wallet, profile in wallet_profiles.items():
        if not isinstance(profile, dict):
            continue

        evaluated_clusters = int(profile.get("evaluated_clusters", 0) or 0)
        clv_observations = int(profile.get("clv_observations", 0) or 0)
        dynamic_weight = float(profile.get("dynamic_weight", 1.0) or 1.0)

        if (
            evaluated_clusters >= 3
            or clv_observations >= 2
            or dynamic_weight >= 1.15
        ):
            filtered.append(wallet)

    return filtered

def apply_wallet_stability_gating(wallet_profiles, candidate_wallets):
    """
    Keep wallets active until they fail the active filter for
    ACTIVE_WALLET_MAX_FAILURES consecutive loops.
    """
    global TRACKED_WALLETS
    global ACTIVE_WALLET_FAILURE_COUNTS

    candidate_set = set(candidate_wallets)
    current_set = set(TRACKED_WALLETS)

    next_wallets = []

    # Always keep wallets that currently pass
    for wallet in candidate_set:
        ACTIVE_WALLET_FAILURE_COUNTS[wallet] = 0
        next_wallets.append(wallet)
```

```
    # For wallets currently active but not passing this loop,
    # keep them alive until they fail too many times in a row.
    for wallet in current_set:
        if wallet in candidate_set:
            continue

        old_failures = ACTIVE_WALLET_FAILURE_COUNTS.get(wallet, 0)
        new_failures = old_failures + 1
        ACTIVE_WALLET_FAILURE_COUNTS[wallet] = new_failures

        if new_failures < ACTIVE_WALLET_MAX_FAILURES:
            next_wallets.append(wallet)

    # Clean up counters for wallets no longer retained
    retained_set = set(next_wallets)
    ACTIVE_WALLET_FAILURE_COUNTS = {
        wallet: count
        for wallet, count in ACTIVE_WALLET_FAILURE_COUNTS.items()
        if wallet in retained_set
    }

    return sorted(set(next_wallets))

def compute_dynamic_wallet_weights(wallet_profiles):
    for wallet, profile in wallet_profiles.items():
        if not isinstance(profile, dict):
            continue

        evaluated_trades = int(profile.get("evaluated_trades", 0) or 0)
        avg_forward_clv = float(profile.get("avg_forward_clv", 0.0) or 0.0)
        positive_clv_rate = float(profile.get("positive_clv_rate", 0.0) or 0.0)
        noise_count = int(profile.get("noise_count", 0) or 0)

        clv_boost = avg_forward_clv / 2.0
        consistency_boost = positive_clv_rate - 0.5
        noise_penalty = min(0.3, noise_count / max(evaluated_trades * 2, 1))

        leader_count = int(profile.get("leader_count", 0) or 0)
        early_count = int(profile.get("early_count", 0) or 0)
        follower_count = int(profile.get("follower_count", 0) or 0)

        total_roles = max(leader_count + early_count + follower_count, 1)

        leader_rate = leader_count / total_roles
```

```
        early_rate = early_count / total_roles
        follower_rate = follower_count / total_roles

        if avg_forward_clv > 0:
            role_score = (
                (leader_rate * 0.4) +
                (early_rate * 0.3) +
                (follower_rate * 0.1)
            )
        else:
            role_score = (
                (leader_rate * -0.3) +
                (early_rate * 0.1) -
                (follower_rate * 0.2)
            )

        raw_score = 1.0 + clv_boost + consistency_boost + role_score - noise_penalty
        confidence = min(1.0, (evaluated_trades + profile.get("clv_observations", 0)) / 30.0)

        dynamic_weight = 1.0 + ((raw_score - 1.0) * confidence)
        dynamic_weight = max(0.75, min(1.5, dynamic_weight))

        profile["confidence"] = round(confidence, 4)
        profile["dynamic_weight"] = round(dynamic_weight, 4)

    return wallet_profiles


GAMMA_MARKET_CACHE = {}


def fetch_gamma_market_metadata(slug, outcome):
    cache_key = (slug, outcome)
    if cache_key in GAMMA_MARKET_CACHE:
        return GAMMA_MARKET_CACHE[cache_key]

    encoded_slug = urllib.parse.quote(slug, safe="")
    url = f"https://gamma-api.polymarket.com/markets?slug={encoded_slug}"

    try:
        req = urllib.request.Request(
            url,
            headers={
                "User-Agent": "Mozilla/5.0",
```

```
            "Accept": "application/json",
        },
    )

    with urllib.request.urlopen(req, timeout=10) as response:
        raw = response.read().decode("utf-8")
        data = json.loads(raw)

except Exception as e:
    print(f"[Gamma lookup error] slug={slug} outcome={outcome} url={url} error={repr(e)}")
    result = {"price": None, "event_start_time": None}
    GAMMA_MARKET_CACHE[cache_key] = result
    return result

if not isinstance(data, list) or not data:
    result = {"price": None, "event_start_time": None}
    GAMMA_MARKET_CACHE[cache_key] = result
    return result

market = data[0]
if not isinstance(market, dict):
    result = {"price": None, "event_start_time": None}
    GAMMA_MARKET_CACHE[cache_key] = result
    return result

outcomes_raw = market.get("outcomes")
prices_raw = market.get("outcomePrices")

event_start_time = (
    market.get("gameStartTime")
    or market.get("game_start_time")
    or market.get("scheduledStart")
    or market.get("scheduled_start")
    or market.get("startTime")
    or market.get("start_time")
    or market.get("startDate")
    or market.get("start_date")
)

events = market.get("events")
if isinstance(events, list) and events:
    first_event = events[0]
    if isinstance(first_event, dict):
        if slug == "norman-powell-points-o-u-195" or "norman-powell" in slug:
```

```
            print("[Gamma market debug] slug:", slug)
            print("[Gamma market debug] market keys:", sorted(list(market.keys())))
            print("[Gamma market debug] first event:", json.dumps(first_event, indent=2)[:2000])

        event_start_time = (
            first_event.get("gameStartTime")
            or first_event.get("game_start_time")
            or first_event.get("scheduledStart")
            or first_event.get("scheduled_start")
            or first_event.get("startTime")
            or first_event.get("start_time")
            or first_event.get("startDate")
            or first_event.get("start_date")
            or event_start_time
        )

    if outcomes_raw is None or prices_raw is None:
        result = {"price": None, "event_start_time": event_start_time}
        GAMMA_MARKET_CACHE[cache_key] = result
        return result

    try:
        outcomes = json.loads(outcomes_raw) if isinstance(outcomes_raw, str) else outcomes_raw
        prices = json.loads(prices_raw) if isinstance(prices_raw, str) else prices_raw
    except Exception as e:
        print(f"[Gamma parse error] slug={slug} outcome={outcome} error={repr(e)}")
        result = {"price": None, "event_start_time": event_start_time}
        GAMMA_MARKET_CACHE[cache_key] = result
        return result

    if not isinstance(outcomes, list) or not isinstance(prices, list):
        result = {"price": None, "event_start_time": event_start_time}
        GAMMA_MARKET_CACHE[cache_key] = result
        return result

    if outcome not in outcomes:
        result = {"price": None, "event_start_time": event_start_time}
        GAMMA_MARKET_CACHE[cache_key] = result
        return result

    idx = outcomes.index(outcome)

    try:
        price = float(prices[idx])
```

```
    except Exception as e:
        print(f"[Gamma price conversion error] slug={slug} outcome={outcome} prices={prices}
error={repr(e)}")
        price = None

    result = {
        "price": price,
        "event_start_time": event_start_time,
    }
    GAMMA_MARKET_CACHE[cache_key] = result
    return result


def attach_position_data_and_score(groups, position_lookup, wallet_medians, wallet_profiles,
fair_price_lookup):
    scored = []

    for g in groups:
        if not isinstance(g, dict):
            continue

        key = (g["wallet"], g["slug"], g["outcome"])
        pos = position_lookup.get(key)

        wallet = g["wallet"]
        median_notional = wallet_medians.get(wallet, 0)

        avg_trade_size = g["total_size"] / max(g["buy_count"], 1)
        avg_trade_price = float(g.get("avg_trade_price", 0) or 0)
        avg_trade_notional = avg_trade_size * avg_trade_price

        # --- NEW: avoid overpriced favorites ---
        MAX_ENTRY_PRICE = 0.80

        if avg_trade_price > MAX_ENTRY_PRICE:
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = "Price too high (poor risk/reward)"
            # --- NEW: market movement (chase) penalty ---
            movement = g.get("market_movement_cents")

            if movement is not None:
                # mild chase
```

```
        if movement > 1.5 and movement <= 3.0:
            g["stake_pct"] = max(int(g.get("stake_pct", 0) * 0.75), 10)
            g["reason"] += " | Chase: mild"

        # moderate chase
        elif movement > 3.0 and movement <= 5.0:
            g["stake_pct"] = max(int(g.get("stake_pct", 0) * 0.5), 5)
            if g.get("label") == "BET":
                g["label"] = "LEAN"
            g["reason"] += " | Chase: moderate"

        # extreme chase → PASS
        elif movement > 5.0:
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = f"Too much market movement (+{movement}c)"
    scored.append(g)
    continue
if median_notional > 0:
    size_ratio = avg_trade_notional / median_notional
else:
    size_ratio = 0

size_ratio = min(size_ratio, 10)

if size_ratio >= 10:
    size_points = 30
elif size_ratio >= 5:
    size_points = 22
elif size_ratio >= 3:
    size_points = 16
elif size_ratio >= 2:
    size_points = 10
elif size_ratio >= 1:
    size_points = 5
elif size_ratio >= 0.5:
    size_points = 2
else:
    size_points = 0

total_notional = float(g.get("total_size", 0) or 0) * avg_trade_price

if total_notional >= 100000:
```

```
    absolute_size_points = 20
elif total_notional >= 50000:
    absolute_size_points = 14
elif total_notional >= 25000:
    absolute_size_points = 10
elif total_notional >= 10000:
    absolute_size_points = 6
elif total_notional >= 5000:
    absolute_size_points = 3
else:
    absolute_size_points = 0

conviction_points = min(size_points + absolute_size_points, 40)

g = dict(g)
g["size_ratio"] = round(size_ratio, 2)
g["size_points"] = size_points
g["absolute_size_points"] = absolute_size_points
g["conviction_points"] = conviction_points
g["total_notional"] = round(total_notional, 2)
g["avg_trade_size"] = round(avg_trade_size, 6)
g["avg_trade_notional"] = round(avg_trade_notional, 2)

sequence_role = str(g.get("sequence_role", "") or "").lower()
buy_count = int(g.get("buy_count", 0) or 0)

min_size_ratio_required = 0.25

if sequence_role in {"leader", "early"}:
    min_size_ratio_required = 0.12
elif buy_count >= 20:
    min_size_ratio_required = 0.10
elif buy_count >= 10:
    min_size_ratio_required = 0.15

if size_ratio < min_size_ratio_required:
    g["current_price"] = None
    g["market_movement_cents"] = None
    g["seconds_since_last_buy"] = None
    g["label"] = "PASS"
    g["score"] = 0
    g["stake_pct"] = 0
    g["reason"] = (
        f"Far below wallet size baseline (soft-filter, "
```

```
        f"ratio={round(size_ratio, 2)}, min={min_size_ratio_required})"
    )
    scored.append(g)
    continue

time_since_last_buy = int(time.time()) - g["last_timestamp"]
g["seconds_since_last_buy"] = time_since_last_buy

gamma_meta = fetch_gamma_market_metadata(g["slug"], g["outcome"])
g["event_start_time"] = gamma_meta.get("event_start_time")

if not pos:
    gamma_price = gamma_meta.get("price")
    g["current_price"] = gamma_price

    if gamma_price is None:
        g["market_movement_cents"] = None
        g["label"] = "WATCH"
        g["score"] = 25
        g["stake_pct"] = 0
        g["reason"] = "No matching current position and no Gamma price found"
        scored.append(g)
        continue

    market_movement_cents = round((gamma_price - avg_trade_price) * 100, 2)
    g["market_movement_cents"] = market_movement_cents

else:
    current_price = float(pos.get("curPrice", 0) or 0)
    g["current_price"] = current_price

    fair_price = fair_price_lookup.get((g["slug"], g["outcome"]))
    if fair_price is None:
        fair_price = avg_trade_price

    g["fair_price"] = round(fair_price, 4)
    fair_american_odds = price_to_american_odds(fair_price)
    g["fair_american_odds"] = fair_american_odds

    wallet_entry_price = avg_trade_price
    g["wallet_entry_price"] = round(wallet_entry_price, 4)

    market_movement_cents = round((current_price - wallet_entry_price) * 100, 2)
    g["market_movement_cents"] = market_movement_cents
```

```
    edge_pct = 0.0
    try:
        if current_price and fair_price and float(current_price) > 0:
            edge_pct = ((float(fair_price) / float(current_price)) - 1.0) * 100.0
    except Exception:
        edge_pct = 0.0

    g["edge_pct"] = round(edge_pct, 2)

# --- DYNAMIC price drift filter: must run after current_price/market movement are set ---
current_price_value = float(g.get("current_price", 0) or 0)
price_drift = current_price_value - avg_trade_price
g["price_drift"] = round(price_drift, 4)

sequence_role = str(g.get("sequence_role", "") or "").lower()
size_ratio = float(g.get("size_ratio", 0) or 0)
buy_count = int(g.get("buy_count", 0) or 0)
signal_age_seconds = int(time_since_last_buy or 999999)

max_price_drift_allowed = 0.025
price_drift_tier = "strict"

if (
    sequence_role in {"leader", "early"}
    and signal_age_seconds <= 120
    and size_ratio >= 0.60
    and buy_count >= 4
):
    max_price_drift_allowed = 0.06
    price_drift_tier = "elite"
elif (
    sequence_role in {"leader", "early"}
    and signal_age_seconds <= 240
    and size_ratio >= 0.45
    and buy_count >= 3
):
    max_price_drift_allowed = 0.05
    price_drift_tier = "strong"
elif (
    sequence_role in {"leader", "early"}
    and signal_age_seconds <= 300
    and size_ratio >= 0.30
):
```

```
    max_price_drift_allowed = 0.045
    price_drift_tier = "moderate"
elif buy_count >= 3 and signal_age_seconds <= 240:
    max_price_drift_allowed = 0.04
    price_drift_tier = "moderate"

g["max_price_drift_allowed"] = round(max_price_drift_allowed, 4)
g["price_drift_tier"] = price_drift_tier

if price_drift > max_price_drift_allowed:
    g["label"] = "PASS"
    g["score"] = 0
    g["stake_pct"] = 0
    g["reason"] = (
        f"Too much price drift (+{round(price_drift * 100, 2)} cents, "
        f"max {round(max_price_drift_allowed * 100, 2)} cents, "
        f"tier={price_drift_tier})"
    )
    scored.append(g)
    continue
elif price_drift > 0.02:
    g["price_drift_flag"] = "moderate"
else:
    g["price_drift_flag"] = "low"

is_live = is_live_market(g)

raw_event = g.get("event") or {}
raw_events = g.get("events") or []
first_event = raw_events[0] if isinstance(raw_events, list) and raw_events else {}

event_start = (
    g.get("event_start_time")
    or g.get("start_time")
    or g.get("game_start_time")
    or g.get("scheduled_time")
    or g.get("event_start")
    or g.get("startDate")
    or g.get("start_date")
    or g.get("startsAt")
    or g.get("startTime")
    or g.get("gameStartTime")
    or g.get("scheduledStart")
    or g.get("scheduled_start")
```

```
    or raw_event.get("start_time")
    or raw_event.get("startTime")
    or raw_event.get("start_date")
    or raw_event.get("startDate")
    or raw_event.get("scheduled_start")
    or raw_event.get("scheduledStart")
    or first_event.get("start_time")
    or first_event.get("startTime")
    or first_event.get("start_date")
    or first_event.get("startDate")
    or first_event.get("scheduled_start")
    or first_event.get("scheduledStart")
    or g.get("end_date")
    or g.get("resolve_time")
)

if not event_start:
    slug = g.get("slug")
    if slug:
        gamma_meta_fallback = fetch_gamma_market_metadata(slug, g.get("outcome"))
        event_start = gamma_meta_fallback.get("event_start_time")
        g["event_start_time"] = event_start

if event_start:
    try:
        from datetime import datetime, timezone

        event_start_dt = datetime.fromisoformat(
            str(event_start).replace("Z", "+00:00")
        )
        now_dt = datetime.now(timezone.utc)

        if event_start_dt <= now_dt:
            is_live = True
    except Exception:
        pass

g["is_live"] = is_live
g["market_phase"] = "Live" if is_live else "Pre-Game"

if is_live:
    if float(g.get("edge_pct", 0) or 0) <= 0:
        g["label"] = "PASS"
        g["score"] = 0
```

```
        g["stake_pct"] = 0
        g["reason"] = f"Live market with no current edge ({g.get('edge_pct', 0)}%)"
        scored.append(g)
        continue

    if int(time_since_last_buy or 999999) > 60:
        g["label"] = "PASS"
        g["score"] = 0
        g["stake_pct"] = 0
        g["reason"] = f"Live signal too stale ({int(time_since_last_buy)}s old)"
        scored.append(g)
        continue
else:
    edge_pct = float(g.get("edge_pct", 0) or 0)
    current_price_for_edge = float(g.get("current_price", 0) or 0)

    if current_price_for_edge >= 0.65:
        min_edge = 0.0
    else:
        min_edge = -2.0

    g["min_pregame_edge_pct"] = min_edge

    if edge_pct < min_edge:
        g["label"] = "PASS"
        g["score"] = 0
        g["stake_pct"] = 0
        g["reason"] = f"Pre-game edge too far gone ({edge_pct}%, min {min_edge}%)"
        scored.append(g)
        continue

if time_since_last_buy <= 120:
    live_age_penalty = 0
    pregame_age_penalty = 0
    age_bucket = "fresh"
elif time_since_last_buy <= 300:
    live_age_penalty = 5
    pregame_age_penalty = 2
    age_bucket = "slightly stale"
elif time_since_last_buy <= 600:
    live_age_penalty = 10
    pregame_age_penalty = 4
    age_bucket = "stale"
elif time_since_last_buy <= 1200:
```

```
    live_age_penalty = 16
    pregame_age_penalty = 7
    age_bucket = "old"
elif time_since_last_buy <= 1800:
    live_age_penalty = 22
    pregame_age_penalty = 10
    age_bucket = "very old"
else:
    live_age_penalty = 35
    pregame_age_penalty = 14
    age_bucket = "dead"

if is_live:
    age_penalty = live_age_penalty + 5
    if age_bucket == "fresh":
        age_bucket = "live-fresh"
    elif age_bucket == "slightly stale":
        age_bucket = "live-slightly-stale"
    elif age_bucket == "stale":
        age_bucket = "live-stale"
    elif age_bucket == "old":
        age_bucket = "live-old"
    else:
        age_bucket = "live-very-old"
    g["age_adjustment"] = "live_full_penalty"
else:
    conviction_points_for_decay = int(g.get("conviction_points", 0) or 0)

    if conviction_points_for_decay >= 30:
        age_penalty = max(pregame_age_penalty - 4, 0)
        g["age_adjustment"] = "pregame_light_penalty_elite_conviction"
    elif conviction_points_for_decay >= 20:
        age_penalty = max(pregame_age_penalty - 2, 0)
        g["age_adjustment"] = "pregame_light_penalty_strong_conviction"
    else:
        age_penalty = pregame_age_penalty
        g["age_adjustment"] = "pregame_light_penalty_standard"

g["age_penalty"] = age_penalty
g["age_bucket"] = age_bucket
g["age_bucket"] = age_bucket

confirmation_count = 0
has_structure_confirmation = False
```

```
if size_points >= 5:
    confirmation_count += 1
if g.get("accumulation_points", 0) >= 20:
    confirmation_count += 1
    has_structure_confirmation = True
if float(g.get("edge_pct", 0) or 0) >= 2.0:
    confirmation_count += 1

g["confirmation_count"] = confirmation_count

is_leader_or_early = sequence_role in {"leader", "early"}
accumulation_points = int(g.get("accumulation_points", 0) or 0)
consensus_weighted = float(g.get("consensus_weighted", 0) or 0)
has_structure = (consensus_weighted >= 1.5 or accumulation_points >= 20)

conviction_points = int(g.get("conviction_points", 0) or 0)
absolute_size_points = int(g.get("absolute_size_points", 0) or 0)

if (
    is_leader_or_early
    and has_structure
    and (
        confirmation_count >= 3
        or (
            confirmation_count >= 2
            and float(g.get("size_ratio", 0) or 0) >= 0.50
            and size_points >= 2
        )
    )
):
    base_label = "BET"
    base_score = 75 + conviction_points
    base_stake_pct = 50
    base_reason = (
        f"Strong signal ({sequence_role}) | Confirmations: {confirmation_count}"
        f" | Conviction={conviction_points}"
    )
elif is_leader_or_early:
    buy_count_for_override = int(g.get("buy_count", 0) or 0)
    size_ratio_for_override = float(g.get("size_ratio", 0) or 0)
    if buy_count_for_override >= 50 and size_ratio_for_override >= 0.7:
        base_label = "BET"
        base_score = 78 + conviction_points
```

```
            base_stake_pct = 60
            base_reason = (
                f"Extreme accumulation ({sequence_role}) | Buys: {buy_count_for_override}"
                f" | Conviction={conviction_points}"
            )
        else:
            base_label = "LEAN"
            base_score = 60 + conviction_points
            base_stake_pct = 30
            base_reason = (
                f"Leader/early but no structure ({sequence_role}) | Confirmations:
{confirmation_count}"
                f" | Conviction={conviction_points}"
            )
    else:
        base_label = "LEAN"
        base_score = 60 + conviction_points
        base_stake_pct = 30
        base_reason = (
            f"Follower signal capped at LEAN | Conviction={conviction_points}"
        )

    edge_pct_for_decay = float(g.get("edge_pct", 0) or 0)
    size_ratio_for_decay = float(g.get("size_ratio", 0) or 0)

    strong_structure = (
        confirmation_count >= 3
        and size_ratio_for_decay >= 0.8
        and int(g.get("buy_count", 0) or 0) >= 3
    )

    decayed_score = max(base_score - age_penalty, 0)

    if (
        not is_live
        and strong_structure
        and edge_pct_for_decay >= -2.0
    ):
        decayed_score = max(decayed_score, base_score - int(age_penalty * 0.5))
        g["time_decay_adjustment"] = "reduced_for_strong_structure"

    strong_low_movement_rescue = False

    if (
```

```
    not is_live
    and base_label == "BET"
    and confirmation_count >= 3
    and float(g.get("size_ratio", 0) or 0) >= 1.0
    and int(g.get("buy_count", 0) or 0) >= 3
    and abs(float(g.get("market_movement_cents", 0) or 0)) <= 3.0
    and int(time_since_last_buy or 999999) <= 2100
):
    strong_low_movement_rescue = True
    decayed_score = max(decayed_score, 72)
    g["decay_rescue"] = "pregame_strong_low_movement"

if base_label == "PASS":
    g["label"] = "PASS"
    g["score"] = 0
    g["stake_pct"] = 0
    g["reason"] = base_reason
    scored.append(g)
    continue

if decayed_score < 60 and not strong_low_movement_rescue:
    g["label"] = "PASS"
    g["score"] = 0
    g["stake_pct"] = 0
    g["reason"] = f"{base_reason} | Decayed below threshold ({age_bucket})"
    scored.append(g)
    continue

g["label"] = base_label
g["score"] = decayed_score

conviction_points_for_stake = int(g.get("conviction_points", 0) or 0)
size_ratio_for_stake = float(g.get("size_ratio", 0) or 0)
total_notional_for_stake = float(g.get("total_notional", 0) or 0)
edge_pct_for_stake = float(g.get("edge_pct", 0) or 0)
current_price_for_stake = float(g.get("current_price", 0) or 0)

# --- base stake from score ---
if decayed_score >= 95:
    base_stake_pct = 125
elif decayed_score >= 88:
    base_stake_pct = 110
elif decayed_score >= 80:
    base_stake_pct = 100
```

```
elif decayed_score >= 72:
    base_stake_pct = 90
else:
    base_stake_pct = 80

# --- conviction bump from size ratio ---
ratio_bump_pct = 0
if size_ratio_for_stake >= 10:
    ratio_bump_pct = 20
elif size_ratio_for_stake >= 5:
    ratio_bump_pct = 15
elif size_ratio_for_stake >= 3:
    ratio_bump_pct = 10
elif size_ratio_for_stake >= 2:
    ratio_bump_pct = 5

# --- conviction bump from absolute notional ---
total_notional_bump_pct = 0
if total_notional_for_stake >= 100000:
    total_notional_bump_pct = 15
elif total_notional_for_stake >= 50000:
    total_notional_bump_pct = 10
elif total_notional_for_stake >= 25000:
    total_notional_bump_pct = 5

conviction_bump_pct = min(ratio_bump_pct + total_notional_bump_pct, 25)
raw_stake_pct = base_stake_pct + conviction_bump_pct

# --- price cap ---
# User preference: allow up to 100% through +150 (price 0.40)
if current_price_for_stake < 0.19:
    price_cap_pct = 50
elif current_price_for_stake < 0.24:
    price_cap_pct = 60
elif current_price_for_stake < 0.29:
    price_cap_pct = 70
elif current_price_for_stake < 0.34:
    price_cap_pct = 80
elif current_price_for_stake < 0.40:
    price_cap_pct = 90
elif current_price_for_stake < 0.50:
    price_cap_pct = 100
elif current_price_for_stake < 0.65:
    price_cap_pct = 110
```

```
elif current_price_for_stake < 0.80:
    price_cap_pct = 125
else:
    price_cap_pct = 110

# --- softer edge cap ---
if edge_pct_for_stake <= -2.0:
    edge_cap_pct = 80
elif edge_pct_for_stake < 0:
    edge_cap_pct = 90
elif edge_pct_for_stake < 1.0:
    edge_cap_pct = 100
elif edge_pct_for_stake < 2.5:
    edge_cap_pct = 110
else:
    edge_cap_pct = 150

final_cap_pct = min(price_cap_pct, edge_cap_pct)
unclipped_stake_pct = max(50, min(raw_stake_pct, final_cap_pct))

# --- snap stake to production buckets ---
allowed_buckets = [50, 60, 70, 80, 90, 100, 110, 125, 150]

def snap_to_bucket(x):
    return min(allowed_buckets, key=lambda b: abs(b - x))

g["stake_pct"] = snap_to_bucket(unclipped_stake_pct)
g["reason"] = (
    f"{base_reason} | Age: {age_bucket}"
    f" | Stake base={base_stake_pct}%"
    f", ratio bump={ratio_bump_pct}%"
    f", notional bump={total_notional_bump_pct}%"
    f", raw={raw_stake_pct}%"
    f", price cap={price_cap_pct}%"
    f", edge cap={edge_cap_pct}%"
    f", final={g['stake_pct']}%"
)

# --- FINAL BET STRUCTURE ENFORCEMENT (GLOBAL) ---
if g.get("label") == "BET":
    confirmation_count = int(g.get("confirmation_count", 0) or 0)
    accumulation_points = int(g.get("accumulation_points", 0) or 0)
    size_ratio = float(g.get("size_ratio", 0) or 0)
    market_movement_cents = float(g.get("market_movement_cents", 0) or 0)
```

```
age_bucket = str(g.get("age_bucket", "") or "").lower()
consensus_upgrade = bool(g.get("consensus_upgrade", False))
fair_price = g.get("fair_price")
edge_pct = g.get("edge_pct")
wallet_entry_price = g.get("wallet_entry_price")
current_price = float(g.get("current_price", 0) or 0)

clv_key = make_clv_key(
    g.get("slug", ""),
    g.get("outcome", ""),
    g.get("wallet", ""),
)
clv_row = clv_tracker.get(clv_key, {}) if isinstance(clv_tracker, dict) else {}
latest_price_for_clv = clv_row.get("latest_price")
entry_price_for_clv = clv_row.get("entry_price")
instant_clv_cents = None

try:
    if latest_price_for_clv is not None and entry_price_for_clv is not None:
        instant_clv_cents = round(
            (float(latest_price_for_clv) - float(entry_price_for_clv)) * 100,
            2,
        )
except Exception:
    instant_clv_cents = None

g["instant_clv_cents"] = instant_clv_cents

max_adverse_instant_clv_cents = -5.0

if (
    int(g.get("stake_pct", 0) or 0) >= 110
    or float(g.get("size_ratio", 0) or 0) >= 1.5
    or float(g.get("total_size", 0) or 0) >= 10000
    or int(g.get("buy_count", 0) or 0) >= 8
    or str(g.get("consensus_type", "") or "").lower() == "full"
):
    max_adverse_instant_clv_cents = -8.0

if (
    int(g.get("stake_pct", 0) or 0) >= 125
    and (
        float(g.get("size_ratio", 0) or 0) >= 2.0
        or float(g.get("total_size", 0) or 0) >= 25000
```

```
            or int(g.get("buy_count", 0) or 0) >= 12
            or str(g.get("consensus_type", "") or "").lower() == "full"
        )
    ):
        max_adverse_instant_clv_cents = -10.0

    g["max_adverse_instant_clv_cents"] = max_adverse_instant_clv_cents

    has_strong_confirmations = confirmation_count >= 2
    has_strong_accumulation = (
        accumulation_points >= 30
        and size_ratio >= 0.25
    )

    if not (has_strong_confirmations or has_strong_accumulation):
        g["label"] = "PASS"
        g["score"] = 0
        g["stake_pct"] = 0
        g["reason"] = (
            f"Final filter: rejected weak structure "
            f"(confirmations={confirmation_count}, accumulation={accumulation_points},
size_ratio={round(size_ratio, 2)})"
        )

    elif fair_price is None or edge_pct is None or wallet_entry_price is None:
        g["label"] = "PASS"
        g["score"] = 0
        g["stake_pct"] = 0
        g["reason"] = "Final filter: missing fair price / edge data for BET"

    else:
        edge_pct = float(edge_pct or 0)

        if current_price >= 0.65:
            min_edge = 0.0
        else:
            min_edge = -2.0

        if edge_pct < min_edge:
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = (
                f"Final filter: edge too low for price "
```

```
                f"(edge={round(edge_pct, 2)}%, min={min_edge}%, price={round(current_price,
3)})"
            )

        elif (
            instant_clv_cents is not None
            and instant_clv_cents < 0
        ):
            g["label"] = "LEAN"
            g["stake_pct"] = min(int(g.get("stake_pct", 0) or 0), 80)
            g["reason"] = (
                f"{g.get('reason', '')} | Downgraded: negative instant CLV "
                f"({instant_clv_cents:+.2f}c)"
            )

        elif (
            instant_clv_cents is not None
            and instant_clv_cents <= max_adverse_instant_clv_cents
        ):
            g["label"] = "LEAN"
            g["stake_pct"] = min(int(g.get("stake_pct", 0) or 0), 80)
            g["reason"] = (
                f"{g.get('reason', '')} | Downgraded: adverse instant CLV "
                f"({instant_clv_cents:+.2f}c, min={max_adverse_instant_clv_cents:+.1f}c)"
            )

        elif (
            not is_live
            and market_movement_cents <= -10.0
        ):
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = (
                f"Final filter: adverse stale move for BET "
                f"(market_movement={round(market_movement_cents, 2)}c)"
            )

        elif (
            not is_live
            and not consensus_upgrade
            and confirmation_count < 3
            and age_bucket in {"old", "very old", "dead"}
            and float(g.get("size_ratio", 0) or 0) < 1.0
```

```
            and abs(float(g.get("market_movement_cents", 0) or 0)) <= 2.0
        ):
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = (
                f"Final filter: 2-confirmation BET too stale "
                f"(age={age_bucket}, confirmations={confirmation_count}, "
                f"size_ratio={round(float(g.get('size_ratio', 0) or 0), 2)}, "
                f"movement={round(abs(float(g.get('market_movement_cents', 0) or 0)), 2)}c)"
            )

    scored.append(g)

return scored

def build_cross_wallet_consensus(accumulation_groups, scored_candidates, wallet_profiles):
    scored_index = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        scored_index[(g["slug"], g["outcome"], g["wallet"])] = g

    consensus = {}

    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue

        key = (g["slug"], g["outcome"])

        if key not in consensus:
            consensus[key] = {
                "slug": g["slug"],
                "outcome": g["outcome"],
                "market": g["title"],
                "wallets_all": set(),
                "wallets_scored": set(),
                "total_size_all": 0,
                "total_size_scored": 0,
                "weighted_wallet_score_all": 0.0,
                "weighted_wallet_score_scored": 0.0,
                "groups": [],
            }
```

```
wallet = g["wallet"]
wallet_weight = float(
    wallet_profiles.get(wallet, {}).get(
        "dynamic_weight",
        WALLET_WEIGHTS.get(wallet, 1.0)
    )
)

if wallet not in consensus[key]["wallets_all"]:
    consensus[key]["weighted_wallet_score_all"] += wallet_weight

consensus[key]["wallets_all"].add(wallet)
consensus[key]["total_size_all"] += g["total_size"]
consensus[key]["groups"].append(g)

scored_match = scored_index.get((g["slug"], g["outcome"], wallet))
if scored_match:

    # --- NEW: wallet quality filter ---
    profile = wallet_profiles.get(wallet, {})
    confidence = float(profile.get("confidence", 0) or 0)
    clv_obs = int(profile.get("clv_observations", 0) or 0)
    pos_clv_rate = float(profile.get("positive_clv_rate", 0) or 0)

    is_trusted_wallet = (
        (clv_obs >= 3 and pos_clv_rate >= 0.55)
        or confidence >= 0.4
    )

    if not is_trusted_wallet:
        continue

    label = scored_match.get("label")
    clv = float(scored_match.get("market_movement_cents", 0) or 0)
    age = int(scored_match.get("seconds_since_last_buy", 999999) or 999999)

    is_actionable = label in {"LEAN", "BET", "STRONG BET"}
    is_recent_negative_clv = (
        label == "PASS"
        and clv >= -1.0
        and age <= 180
    )
```

```
        if is_actionable or is_recent_negative_clv:
            if wallet not in consensus[key]["wallets_scored"]:
                consensus[key]["weighted_wallet_score_scored"] += wallet_weight

            consensus[key]["wallets_scored"].add(wallet)
            consensus[key]["total_size_scored"] += scored_match.get("total_size", 0)

consensus_list = []

for _, c in consensus.items():
    c["wallet_count_all"] = len(c["wallets_all"])
    c["wallet_count_scored"] = len(c["wallets_scored"])
    c["weighted_wallet_score_all"] = round(c["weighted_wallet_score_all"], 2)
    c["weighted_wallet_score_scored"] = round(c["weighted_wallet_score_scored"], 2)

    score = 0
    tag = "none"

    valid_roles = 0
    for g in c["groups"]:
        role = g.get("sequence_role")
        if role in {"leader", "early"}:
            valid_roles += 1

    if c["wallet_count_scored"] >= 2 and valid_roles >= 1:
        tag = "full"
        score += 50

        if c["wallet_count_scored"] >= 3:
            score += 25

        if c["weighted_wallet_score_scored"] >= 2.0:
            score += 10
        if c["weighted_wallet_score_scored"] >= 2.5:
            score += 10
        if c["weighted_wallet_score_scored"] >= 3.0:
            score += 10

        if c["total_size_scored"] > 5000:
            score += 10
        if c["total_size_scored"] > 15000:
            score += 15

    elif c["wallet_count_all"] >= 2 and c["wallet_count_scored"] == 1 and valid_roles >= 1:
```

```
            tag = "near"
            score += 25

            if c["weighted_wallet_score_all"] >= 2.0:
                score += 5
            if c["weighted_wallet_score_all"] >= 2.5:
                score += 5

            if c["total_size_all"] > 5000:
                score += 10
            if c["total_size_all"] > 15000:
                score += 15

        else:
            continue

        c["consensus_type"] = tag
        c["consensus_score"] = score
        consensus_list.append(c)

    consensus_list.sort(key=lambda x: x["consensus_score"], reverse=True)
    return consensus_list


def apply_consensus_upgrades(scored_candidates, consensus_list, wallet_profiles):
    consensus_lookup = {}
    for c in consensus_list:
        if not isinstance(c, dict):
            continue
        key = (c.get("slug", ""), c.get("outcome", ""))
        consensus_lookup[key] = c

    scored_index = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        scored_index[(g.get("slug", ""), g.get("outcome", ""), g.get("wallet", ""))] = g

    upgraded = []

    consensus_debug = {
        "total_candidates": 0,
        "not_dict": 0,
        "no_consensus": 0,
```

```
    "bad_role": 0,
    "bad_label": 0,
    "not_full_consensus": 0,
    "too_few_wallets_scored": 0,
    "weighted_score_too_low": 0,
    "no_leader_or_early": 0,
    "quality_contributors_too_low": 0,
    "size_ratio_too_low": 0,
    "missing_edge": 0,
    "edge_too_low": 0,
    "market_movement_negative": 0,
    "too_old": 0,
    "consensus_score_too_low": 0,
    "upgraded_to_bet": 0,
}

for g in scored_candidates:
    consensus_debug["total_candidates"] += 1

    if not isinstance(g, dict):
        consensus_debug["not_dict"] += 1
        upgraded.append(g)
        continue

    g = dict(g)
    g["consensus_upgrade"] = False
    g["consensus_type"] = None
    g["consensus_score"] = 0
    g["weighted_wallet_score_scored"] = 0
    g["wallet_count_scored"] = 0
    g["wallet_count_all"] = 0

    consensus = consensus_lookup.get((g.get("slug", ""), g.get("outcome", "")))
    if not consensus:
        consensus_debug["no_consensus"] += 1
        upgraded.append(g)
        continue

    g["consensus_type"] = consensus.get("consensus_type")
    g["consensus_score"] = consensus.get("consensus_score", 0)
    g["weighted_wallet_score_scored"] = consensus.get("weighted_wallet_score_scored", 0)
    g["wallet_count_scored"] = consensus.get("wallet_count_scored", 0)
    g["wallet_count_all"] = consensus.get("wallet_count_all", 0)
```

```
        self_role = str(g.get("sequence_role", "") or "").lower()
        if self_role not in {"leader", "early"}:
            consensus_debug["bad_role"] += 1
            upgraded.append(g)
            continue

        if g.get("label") not in {"LEAN", "BET"}:
            upgraded.append(g)
            continue

        # NEW: require minimum base structure before allowing consensus upgrade
        confirmation_count = int(g.get("confirmation_count", 0) or 0)
        size_ratio = float(g.get("size_ratio", 0) or 0)
        accumulation_points = int(g.get("accumulation_points", 0) or 0)

        has_min_structure = (
            confirmation_count >= 2
            or size_ratio >= 0.5
            or accumulation_points >= 30
        )

        if not has_min_structure:
            consensus_debug["no_base_structure"] = consensus_debug.get("no_base_structure",
0) + 1
            upgraded.append(g)
            continue
        if consensus.get("consensus_type") != "full":
            consensus_debug["not_full_consensus"] += 1
            upgraded.append(g)
            continue

        if consensus.get("wallet_count_scored", 0) < 2:
            consensus_debug["too_few_wallets_scored"] += 1
            upgraded.append(g)
            continue

        if float(consensus.get("weighted_wallet_score_scored", 0) or 0) <
CONSENSUS_UPGRADE_MIN_WEIGHTED_SCORE:
            consensus_debug["weighted_score_too_low"] += 1
            upgraded.append(g)
            continue

        consensus_groups = consensus.get("groups", [])
        quality_contributor_count = 0
```

```
leader_or_early_count = 0

for group_row in consensus_groups:
    wallet = group_row.get("wallet", "")
    scored_match = scored_index.get((g.get("slug", ""), g.get("outcome", ""), wallet))
    if not scored_match:
        continue

    wallet_weight = float(wallet_profiles.get(wallet, {}).get("dynamic_weight", 1.0) or 1.0)
    label = str(scored_match.get("label", "") or "").upper()
    role = str(group_row.get("sequence_role", "") or "").lower()
    paired_recent = bool(group_row.get("paired_recent", False))

    if role in {"leader", "early"}:
        leader_or_early_count += 1

    if (
        label in {"LEAN", "BET", "STRONG BET"}
        and wallet_weight >= 1.0
        and role in {"leader", "early"}
        and not paired_recent
    ):
        quality_contributor_count += 1

if leader_or_early_count < 1:
    consensus_debug["no_leader_or_early"] += 1
    upgraded.append(g)
    continue

size_ratio = float(g.get("size_ratio", 0) or 0)
if quality_contributor_count < 2:
    consensus_debug["quality_contributors_too_low"] += 1
    upgraded.append(g)
    continue

size_ratio = float(g.get("size_ratio", 0) or 0)
buy_count = int(g.get("buy_count", 0) or 0)
sequence_role = str(g.get("sequence_role", "") or "").lower()

min_size_ratio_required = 0.3

# leaders / early get more flexibility
if sequence_role in {"leader", "early"}:
    min_size_ratio_required = 0.15
```

```
    # strong accumulation override
    elif buy_count >= 20:
        min_size_ratio_required = 0.10

    elif buy_count >= 10:
        min_size_ratio_required = 0.15

    if size_ratio < min_size_ratio_required:
        consensus_debug["size_ratio_too_low"] += 1
        upgraded.append(g)
        continue

    edge_pct_raw = g.get("edge_pct")
    if edge_pct_raw is None:
        consensus_debug["missing_edge"] += 1
        upgraded.append(g)
        continue

    edge_pct = float(edge_pct_raw or 0)
    consensus_score = int(consensus.get("consensus_score", 0) or 0)
    weighted_wallet_score = float(consensus.get("weighted_wallet_score_scored", 0) or 0)
    seconds_since_last_buy = int(g.get("seconds_since_last_buy", 999999) or 999999)

    dynamic_min_edge_pct = -2.0

    if (
        quality_contributor_count >= 3
        and consensus_score >= 85
        and weighted_wallet_score >=
max(CONSENSUS_UPGRADE_MIN_WEIGHTED_SCORE + 0.5, 2.0)
        and seconds_since_last_buy <=
min(CONSENSUS_UPGRADE_MAX_AGE_SECONDS, 180)
        and size_ratio >= 0.5
    ):
        dynamic_min_edge_pct = -3.5
    elif (
        quality_contributor_count >= 2
        and consensus_score >= 75
        and weighted_wallet_score >= CONSENSUS_UPGRADE_MIN_WEIGHTED_SCORE
        and seconds_since_last_buy <= CONSENSUS_UPGRADE_MAX_AGE_SECONDS
        and size_ratio >= 0.3
    ):
        dynamic_min_edge_pct = -2.0
```

```
else:
    dynamic_min_edge_pct = -0.5

g["consensus_dynamic_min_edge_pct"] = dynamic_min_edge_pct

if edge_pct < dynamic_min_edge_pct:
    consensus_debug["edge_too_low"] += 1
    upgraded.append(g)
    continue

market_movement_cents = float(g.get("market_movement_cents", 0) or 0)
if market_movement_cents < 0.0:
    consensus_debug["market_movement_negative"] += 1
    upgraded.append(g)
    continue

if seconds_since_last_buy > CONSENSUS_UPGRADE_MAX_AGE_SECONDS:
    consensus_debug["too_old"] += 1
    upgraded.append(g)
    continue

if consensus_score < 70:
    consensus_debug["consensus_score_too_low"] += 1
    upgraded.append(g)
    continue

old_reason = g.get("reason", "")
old_score = int(g.get("score", 0) or 0)
old_stake = int(g.get("stake_pct", 0) or 0)

g["label"] = "BET"
g["score"] = max(old_score, 80, consensus_score)
g["stake_pct"] = max(old_stake, 70)
g["consensus_upgrade"] = True
g["reason"] = (
    f"{old_reason} | Upgraded by full cross-wallet consensus "
    f"(wallets_scored={consensus.get('wallet_count_scored', 0)}, "
    f"weighted_score={consensus.get('weighted_wallet_score_scored', 0)}, "
    f"consensus_score={consensus_score}, "
    f"quality_contributors={quality_contributor_count}, "
    f"min_edge={dynamic_min_edge_pct}, "
    f"actual_edge={edge_pct})"
)
```

```
        consensus_debug["upgraded_to_bet"] += 1
        upgraded.append(g)

    print("CONSENSUS UPGRADE DEBUG")
    print("-" * 80)
    print(f"Total candidates checked:          {consensus_debug['total_candidates']}")
    print(f"Missing consensus:                 {consensus_debug['no_consensus']}")
    print(f"Bad role:                          {consensus_debug['bad_role']}")
    print(f"Bad label:                         {consensus_debug['bad_label']}")
    print(f"Not full consensus:                {consensus_debug['not_full_consensus']}")
    print(f"Too few wallets scored:            {consensus_debug['too_few_wallets_scored']}")
    print(f"Weighted score too low:            {consensus_debug['weighted_score_too_low']}")
    print(f"No leader/early contributor:       {consensus_debug['no_leader_or_early']}")
    print(f"Quality contributors too low:      {consensus_debug['quality_contributors_too_low']}")
    print(f"Size ratio too low:                {consensus_debug['size_ratio_too_low']}")
    print(f"Missing edge:                      {consensus_debug['missing_edge']}")
    print(f"Edge too low:                      {consensus_debug['edge_too_low']}")
    print(f"Negative market movement:
{consensus_debug['market_movement_negative']}")
    print(f"Too old:                           {consensus_debug['too_old']}")
    print(f"Consensus score too low:           {consensus_debug['consensus_score_too_low']}")
    print(f"Upgraded to BET:                   {consensus_debug['upgraded_to_bet']}")
    print("-" * 80)

    return upgraded
    consensus_lookup = {}
    for c in consensus_list:
        if not isinstance(c, dict):
            continue
        key = (c.get("slug", ""), c.get("outcome", ""))
        consensus_lookup[key] = c

    scored_index = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        scored_index[(g.get("slug", ""), g.get("outcome", ""), g.get("wallet", ""))] = g

    upgraded = []

    for g in scored_candidates:
        if not isinstance(g, dict):
            upgraded.append(g)
            continue
```

```
        g = dict(g)
        g["consensus_upgrade"] = False
        g["consensus_type"] = None
        g["consensus_score"] = 0
        g["weighted_wallet_score_scored"] = 0

        consensus = consensus_lookup.get((g.get("slug", ""), g.get("outcome", "")))
        if not consensus:
            upgraded.append(g)
            continue

        g["consensus_type"] = consensus.get("consensus_type")
        g["consensus_score"] = consensus.get("consensus_score", 0)
        g["weighted_wallet_score_scored"] = consensus.get("weighted_wallet_score_scored", 0)

        if g.get("label") != "LEAN":
            upgraded.append(g)
            continue

        self_role = str(g.get("sequence_role", "") or "").lower()
        if self_role not in {"leader", "early"}:
            upgraded.append(g)
            continue

        if consensus.get("consensus_type") != "full":
            upgraded.append(g)
            continue

        if consensus.get("wallet_count_scored", 0) < 2:
            upgraded.append(g)
            continue

        if float(consensus.get("weighted_wallet_score_scored", 0) or 0) <
CONSENSUS_UPGRADE_MIN_WEIGHTED_SCORE:
            upgraded.append(g)
            continue

        # --- NEW: require quality contributors for consensus upgrade ---
        consensus_groups = consensus.get("groups", [])
        quality_contributor_count = 0

        for group_row in consensus_groups:
            wallet = group_row.get("wallet", "")
```

```
    scored_match = scored_index.get((g.get("slug", ""), g.get("outcome", ""), wallet))
    if not scored_match:
        continue

    wallet_weight = float(wallet_profiles.get(wallet, {}).get("dynamic_weight", 1.0) or 1.0)
    label = str(scored_match.get("label", "") or "").upper()
    role = str(group_row.get("sequence_role", "") or "")
    paired_recent = bool(group_row.get("paired_recent", False))

    if (
        label in {"LEAN", "BET", "STRONG BET"}
        and wallet_weight >= 1.0
        and role in {"leader", "early"}
        and not paired_recent
    ):
        quality_contributor_count += 1

if quality_contributor_count < 2:
    upgraded.append(g)
    continue

market_movement_cents = float(g.get("market_movement_cents", 0) or 0)
if market_movement_cents < 0:
    upgraded.append(g)
    continue

seconds_since_last_buy = int(g.get("seconds_since_last_buy", 999999) or 999999)
if seconds_since_last_buy > CONSENSUS_UPGRADE_MAX_AGE_SECONDS:
    upgraded.append(g)
    continue

old_reason = g.get("reason", "")
old_score = int(g.get("score", 0) or 0)
old_stake = int(g.get("stake_pct", 0) or 0)

g["label"] = "BET"
g["score"] = max(old_score, 80)
g["stake_pct"] = max(old_stake, 100)
g["consensus_upgrade"] = True
g["reason"] = (
    f"{old_reason} | Upgraded by full cross-wallet consensus "
    f"(wallets_scored={consensus.get('wallet_count_scored', 0)}, "
    f"weighted_score={consensus.get('weighted_wallet_score_scored', 0)}, "
    f"quality_contributors={quality_contributor_count})"
```

```
        )

        upgraded.append(g)

    return upgraded


def build_consensus_diagnostics(accumulation_groups, scored_candidates):
    scored_index = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        scored_index[(g["slug"], g["outcome"], g["wallet"])] = g

    grouped = defaultdict(list)
    for g in accumulation_groups:
        if not isinstance(g, dict):
            continue
        key = (g["slug"], g["outcome"])
        grouped[key].append(g)

    diagnostics = []

    for (slug, outcome), groups in grouped.items():
        wallets = {g["wallet"] for g in groups if isinstance(g, dict)}
        if len(wallets) < 2:
            continue

        market = groups[0].get("title", "")
        rows = []
        actionable_or_near_actionable = False

        for g in sorted(groups, key=lambda x: x.get("total_size", 0), reverse=True):
            wallet = g["wallet"]
            scored_match = scored_index.get((slug, outcome, wallet))

            if scored_match:
                label = scored_match.get("label")
                reason = scored_match.get("reason")
                market_movement_cents = scored_match.get("market_movement_cents")
                seconds_since_last_buy = scored_match.get("seconds_since_last_buy")
                size_ratio = scored_match.get("size_ratio")

                if label in {"LEAN", "BET", "STRONG BET", "PASS"}:
```

```
            actionable_or_near_actionable = True

        rows.append({
            "wallet": wallet,
            "buy_count": g.get("buy_count"),
            "total_size": g.get("total_size"),
            "avg_trade_price": g.get("avg_trade_price"),
            "sequence_role": g.get("sequence_role", "N/A"),
            "label": label,
            "reason": reason,
            "market_movement_cents": market_movement_cents,
            "seconds_since_last_buy": seconds_since_last_buy,
            "size_ratio": size_ratio,
        })
    else:
        if g.get("paired_recent"):
            fail_reason = "Filtered out before scoring - paired recent activity"
        elif g.get("buy_count", 0) < 2:
            fail_reason = "Filtered out before scoring - fewer than 2 buys"
        elif not is_actionable_accumulation_group(g):
            fail_reason = "Filtered out before scoring - avg trade price outside actionable
range"
        else:
            fail_reason = "Not present in scored_candidates - check pipeline filters"

        rows.append({
            "wallet": wallet,
            "buy_count": g.get("buy_count"),
            "total_size": g.get("total_size"),
            "avg_trade_price": g.get("avg_trade_price"),
            "sequence_role": g.get("sequence_role", "N/A"),
            "label": "FILTERED",
            "reason": fail_reason,
            "market_movement_cents": None,
            "seconds_since_last_buy": None,
            "size_ratio": None,
        })

all_paired_filtered = all(
    row.get("label") == "FILTERED"
    and row.get("reason") == "Filtered out before scoring - paired recent activity"
    for row in rows
)
```

```
        if all_paired_filtered:
            continue

        if not actionable_or_near_actionable:
            continue

        diagnostics.append({
            "slug": slug,
            "outcome": outcome,
            "market": market,
            "wallet_count": len(wallets),
            "rows": rows,
        })

    diagnostics.sort(
        key=lambda d: (
            d["wallet_count"],
            sum((r.get("total_size") or 0) for r in d["rows"])
        ),
        reverse=True
    )
    return diagnostics


def get_bet_alert_key(g):
    if not isinstance(g, dict):
        return ""

    slug = str(g.get("slug", "") or "").strip()
    outcome = str(g.get("outcome", "") or "").strip()

    if not slug or not outcome:
        return ""

    return f"{slug}||{outcome}"

def make_clv_key(slug, outcome, wallet):
    return f"{slug}||{outcome}||{wallet}"

def load_clv_tracker():
    try:
        with open(CLV_TRACKER_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
        if isinstance(data, dict):
```

```
        return data
    except Exception:
        pass
    return {}

def save_clv_tracker(clv_tracker):
    try:
        os.makedirs(os.path.dirname(CLV_TRACKER_PATH), exist_ok=True)
        with open(CLV_TRACKER_PATH, "w", encoding="utf-8") as f:
            json.dump(clv_tracker, f, indent=2, sort_keys=True)
    except Exception as e:
        print(f"[CLV tracker save error] {repr(e)}")

def make_tracked_bet_key(g, now_ts):
    if not isinstance(g, dict):
        return None

    slug = str(g.get("slug", "") or "").strip()
    outcome = str(g.get("outcome", "") or "").strip()
    wallet = str(g.get("wallet", "") or "").strip().lower()

    if not slug or not outcome or not wallet:
        return None

    return f"{slug}||{outcome}||{wallet}||{int(now_ts)}"

def load_tracked_bets():
    try:
        with open(TRACKED_BETS_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
            if isinstance(data, dict):
                return data
    except Exception:
        pass
    return {}


def load_alerted_bets():
    try:
        with open(ALERTED_BETS_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
            if isinstance(data, dict):
                return data
    except Exception:
```

```
        pass
    return {}


def save_alerted_bets(alerted_bets):
    try:
        now_ts = int(time.time())
        ttl_seconds = 12 * 60 * 60  # 12 hours
        cleaned = {}
        for key, row in alerted_bets.items():
            if not isinstance(row, dict):
                continue
            last_ts = int(row.get("last_alert_ts", 0) or 0)
            if last_ts and (now_ts - last_ts) <= ttl_seconds:
                cleaned[key] = row

        os.makedirs(os.path.dirname(ALERTED_BETS_PATH), exist_ok=True)
        with open(ALERTED_BETS_PATH, "w", encoding="utf-8") as f:
            json.dump(cleaned, f, indent=2, sort_keys=True)
    except Exception as e:
        print(f"[Alerted bets save error] {repr(e)}")


def save_tracked_bets(tracked_bets):
    try:
        os.makedirs(os.path.dirname(TRACKED_BETS_PATH), exist_ok=True)
        with open(TRACKED_BETS_PATH, "w", encoding="utf-8") as f:
            json.dump(tracked_bets, f, indent=2, sort_keys=True)
    except Exception as e:
        print(f"[Tracked bets save error] {repr(e)}")

def load_signal_metrics_history():
    try:
        with open(SIGNAL_METRICS_HISTORY_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
            if isinstance(data, list):
                return data
    except Exception:
        pass
    return []


def save_signal_metrics_history(signal_metrics_history):
    try:
```

```
        os.makedirs(os.path.dirname(SIGNAL_METRICS_HISTORY_PATH), exist_ok=True)
        with open(SIGNAL_METRICS_HISTORY_PATH, "w", encoding="utf-8") as f:
            json.dump(signal_metrics_history, f, indent=2)
    except Exception as e:
        print(f"[Signal metrics history save error] {repr(e)}")

def record_tracked_bet(g, tracked_bets, now_ts):
    if not isinstance(g, dict) or not isinstance(tracked_bets, dict):
        return

    tracked_bet_key = make_tracked_bet_key(g, now_ts)
    if not tracked_bet_key:
        return

    if tracked_bet_key in tracked_bets:
        return

    tracked_bets[tracked_bet_key] = {
        "tracked_bet_key": tracked_bet_key,
        "alert_ts": int(now_ts),
        "wallet": str(g.get("wallet", "") or "").strip().lower(),
        "slug": str(g.get("slug", "") or "").strip(),
        "title": str(g.get("title", "") or "").strip(),
        "market": str(g.get("market", "") or g.get("title", "") or "").strip(),
        "outcome": str(g.get("outcome", "") or "").strip(),
        "label": str(g.get("label", "") or "").strip(),
        "stake_pct": int(g.get("stake_pct", 0) or 0),
        "score": int(g.get("score", 0) or 0),
        "buy_count": int(g.get("buy_count", 0) or 0),
        "followers": int(get_follower_count(g) or 0),
        "sequence_role": str(g.get("sequence_role", "") or "").strip(),
        "consensus_type": str(g.get("consensus_type", "") or "").strip(),
        "consensus_score": int(g.get("consensus_score", 0) or 0),
        "size_ratio": float(g.get("size_ratio", 0) or 0),
        "total_size": float(g.get("total_size", 0) or 0),
        "avg_trade_price": g.get("avg_trade_price"),
        "wallet_entry_price": g.get("wallet_entry_price"),
        "current_price_at_alert": g.get("current_price"),
        "fair_price_at_alert": g.get("fair_price"),
        "edge_pct_at_alert": g.get("edge_pct"),
        "market_movement_cents_at_alert": g.get("market_movement_cents"),
        "instant_clv_cents_at_alert": g.get("instant_clv_cents"),
        "event_start_time": g.get("event_start_time"),
        "market_phase": g.get("market_phase"),
```

```
        "reason": str(g.get("reason", "") or "").strip(),
        "resolved": False,
        "result": None,
        "winning_outcome": None,
        "resolved_ts": None,
        "resolution_price": None,
    }

def record_signal_metrics_row(g, signal_metrics_history, now_ts, wallet_profiles):
    if not isinstance(g, dict) or not isinstance(signal_metrics_history, list):
        return

    wallet = str(g.get("wallet", "") or "").strip().lower()
    wallet_profile = {}
    if isinstance(wallet_profiles, dict):
        wallet_profile = wallet_profiles.get(wallet, {}) or {}

    try:
        leaderboard_roi = wallet_profile.get("leaderboard_roi")
        if leaderboard_roi is not None:
            leaderboard_roi = float(leaderboard_roi)
    except Exception:
        leaderboard_roi = None

    try:
        total_notional = float(g.get("total_notional", 0) or 0)
    except Exception:
        total_notional = 0.0

    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0

    try:
        size_ratio = float(g.get("size_ratio", 0) or 0)
    except Exception:
        size_ratio = 0.0

    try:
        stake_pct = int(g.get("stake_pct", 0) or 0)
    except Exception:
        stake_pct = 0
```

```
try:
    score = int(g.get("score", 0) or 0)
except Exception:
    score = 0

try:
    edge_pct = float(g.get("edge_pct", 0) or 0)
except Exception:
    edge_pct = 0.0

try:
    current_price = float(g.get("current_price", 0) or 0)
except Exception:
    current_price = 0.0

try:
    wallet_entry_price = float(g.get("wallet_entry_price", 0) or 0)
except Exception:
    wallet_entry_price = 0.0

try:
    market_movement_cents = float(g.get("market_movement_cents", 0) or 0)
except Exception:
    market_movement_cents = 0.0

try:
    followers = int(get_follower_count(g) or 0)
except Exception:
    followers = 0

try:
    consensus_score = int(g.get("consensus_score", 0) or 0)
except Exception:
    consensus_score = 0

try:
    buy_count = int(g.get("buy_count", 0) or 0)
except Exception:
    buy_count = 0

try:
    seconds_since_last_buy = int(g.get("seconds_since_last_buy", 0) or 0)
except Exception:
    seconds_since_last_buy = 0
```

```
    event_start_time = g.get("event_start_time")
    minutes_to_start = get_minutes_to_start(event_start_time, now_ts)
    time_to_start_bucket = get_time_to_start_bucket(minutes_to_start)

    row = {
        "ts": int(now_ts),
        "wallet": wallet,
        "slug": str(g.get("slug", "") or "").strip(),
        "title": str(g.get("title", "") or "").strip(),
        "market": str(g.get("market", "") or g.get("title", "") or "").strip(),
        "outcome": str(g.get("outcome", "") or "").strip(),
        "market_phase": str(g.get("market_phase", "") or "").strip(),
        "label": str(g.get("label", "") or "").strip(),
        "stake_pct": stake_pct,
        "score": score,
        "size_ratio": round(size_ratio, 4),
        "total_size": round(total_size, 4),
        "total_notional": round(total_notional, 2),
        "leaderboard_roi": leaderboard_roi,
        "edge_pct": round(edge_pct, 4),
        "current_price": round(current_price, 6) if current_price else None,
        "wallet_entry_price": round(wallet_entry_price, 6) if wallet_entry_price else None,
        "market_movement_cents": round(market_movement_cents, 4),
        "followers": followers,
        "consensus_type": str(g.get("consensus_type", "") or "").strip(),
        "consensus_score": consensus_score,
        "buy_count": buy_count,
        "age_bucket": str(g.get("age_bucket", "") or "").strip(),
        "seconds_since_last_buy": seconds_since_last_buy,
        "sequence_role": str(g.get("sequence_role", "") or "").strip(),
        "event_start_time": event_start_time,
        "minutes_to_start": minutes_to_start,
        "time_to_start_bucket": time_to_start_bucket,
    }

    signal_metrics_history.append(row)

    max_rows = 5000
    if len(signal_metrics_history) > max_rows:
        del signal_metrics_history[:-max_rows]

TRACKED_BET_RESOLUTION_CACHE = {}
```

```
def fetch_gamma_market_resolution(slug):
    cache_key = str(slug or "").strip().lower()
    if not cache_key:
        return {
            "resolved": False,
            "winning_outcome": None,
            "resolution_price": None,
        }

    if cache_key in TRACKED_BET_RESOLUTION_CACHE:
        return TRACKED_BET_RESOLUTION_CACHE[cache_key]

    encoded_slug = urllib.parse.quote(slug, safe="")
    url = f"https://gamma-api.polymarket.com/markets?slug={encoded_slug}"

    result = {
        "resolved": False,
        "winning_outcome": None,
        "resolution_price": None,
    }

    try:
        data = fetch_json_url(url)
    except Exception:
        TRACKED_BET_RESOLUTION_CACHE[cache_key] = result
        return result

    if not isinstance(data, list) or not data:
        TRACKED_BET_RESOLUTION_CACHE[cache_key] = result
        return result

    market = data[0]
    if not isinstance(market, dict):
        TRACKED_BET_RESOLUTION_CACHE[cache_key] = result
        return result

    outcomes_raw = market.get("outcomes")
    prices_raw = market.get("outcomePrices")

    try:
        outcomes = json.loads(outcomes_raw) if isinstance(outcomes_raw, str) else outcomes_raw
        prices = json.loads(prices_raw) if isinstance(prices_raw, str) else prices_raw
    except Exception:
```

```
        outcomes = None
        prices = None

    if not isinstance(outcomes, list) or not isinstance(prices, list):
        TRACKED_BET_RESOLUTION_CACHE[cache_key] = result
        return result

    is_closed = bool(market.get("closed", False))
    is_active = market.get("active")
    accepting_orders = market.get("acceptingOrders")

    resolved_hint = (
        is_closed
        or is_active is False
        or accepting_orders is False
    )

    winning_outcome = None
    resolution_price = None

    for outcome_name, outcome_price in zip(outcomes, prices):
        try:
            price_value = float(outcome_price)
        except Exception:
            continue

        if price_value >= 0.999:
            winning_outcome = str(outcome_name)
            resolution_price = price_value
            break

    if resolved_hint and winning_outcome is not None:
        result = {
            "resolved": True,
            "winning_outcome": winning_outcome,
            "resolution_price": resolution_price,
        }

    TRACKED_BET_RESOLUTION_CACHE[cache_key] = result
    return result


def update_tracked_bet_results(tracked_bets, now_ts):
    if not isinstance(tracked_bets, dict):
```

```
    return {
        "tracked": 0,
        "resolved": 0,
        "wins": 0,
        "losses": 0,
        "newly_resolved": 0,
    }

tracked = 0
resolved = 0
wins = 0
losses = 0
newly_resolved = 0

for tracked_bet_key, row in tracked_bets.items():
    if not isinstance(row, dict):
        continue

    tracked += 1

    if row.get("resolved"):
        resolved += 1
        if row.get("result") == "WIN":
            wins += 1
        elif row.get("result") == "LOSS":
            losses += 1
        continue

    slug = str(row.get("slug", "") or "").strip()
    tracked_outcome = str(row.get("outcome", "") or "").strip()

    if not slug or not tracked_outcome:
        continue

    resolution = fetch_gamma_market_resolution(slug)
    if not resolution.get("resolved"):
        continue

    winning_outcome = str(resolution.get("winning_outcome", "") or "").strip()
    resolution_price = resolution.get("resolution_price")

    row["resolved"] = True
    row["winning_outcome"] = winning_outcome
    row["resolved_ts"] = int(now_ts)
```

```
        row["resolution_price"] = resolution_price
        row["result"] = "WIN" if tracked_outcome == winning_outcome else "LOSS"

        newly_resolved += 1
        resolved += 1

        if row["result"] == "WIN":
            wins += 1
        else:
            losses += 1

    return {
        "tracked": tracked,
        "resolved": resolved,
        "wins": wins,
        "losses": losses,
        "newly_resolved": newly_resolved,
    }


def summarize_tracked_bets_by_wallet(tracked_bets):
    summary = defaultdict(lambda: {
        "wallet": "",
        "tracked": 0,
        "resolved": 0,
        "wins": 0,
        "losses": 0,
        "avg_edge_pct_at_alert": 0.0,
        "avg_instant_clv_cents_at_alert": 0.0,
    })

    for _, row in tracked_bets.items():
        if not isinstance(row, dict):
            continue

        wallet = str(row.get("wallet", "") or "").strip().lower()
        if not wallet:
            continue

        bucket = summary[wallet]
        bucket["wallet"] = wallet
        bucket["tracked"] += 1

        try:
```

```
            edge_pct_at_alert = float(row.get("edge_pct_at_alert", 0) or 0)
        except Exception:
            edge_pct_at_alert = 0.0

        try:
            instant_clv_cents_at_alert = float(row.get("instant_clv_cents_at_alert", 0) or 0)
        except Exception:
            instant_clv_cents_at_alert = 0.0

        old_n = bucket["tracked"] - 1
        new_n = bucket["tracked"]

        bucket["avg_edge_pct_at_alert"] = round(
            ((bucket["avg_edge_pct_at_alert"] * old_n) + edge_pct_at_alert) / max(new_n, 1),
            2,
        )
        bucket["avg_instant_clv_cents_at_alert"] = round(
            ((bucket["avg_instant_clv_cents_at_alert"] * old_n) + instant_clv_cents_at_alert) /
max(new_n, 1),
            2,
        )

        if row.get("resolved"):
            bucket["resolved"] += 1
            if row.get("result") == "WIN":
                bucket["wins"] += 1
            elif row.get("result") == "LOSS":
                bucket["losses"] += 1

    summary_rows = list(summary.values())

    for row in summary_rows:
        resolved_count = int(row.get("resolved", 0) or 0)
        wins_count = int(row.get("wins", 0) or 0)
        if resolved_count > 0:
            row["win_rate_pct"] = round((wins_count / resolved_count) * 100, 1)
        else:
            row["win_rate_pct"] = None

    summary_rows.sort(
        key=lambda x: (
            int(x.get("resolved", 0) or 0),
            int(x.get("wins", 0) or 0),
            float(x.get("avg_edge_pct_at_alert", 0) or 0),
```

```
        ),
        reverse=True,
    )

    return summary_rows


def record_clv_bet(g, clv_tracker, now_ts):
    if not isinstance(g, dict):
        return

    slug = str(g.get("slug", "") or "").strip()
    outcome = str(g.get("outcome", "") or "").strip()
    wallet = str(g.get("wallet", "") or "").strip().lower()

    if not slug or not outcome or not wallet:
        return

    try:
        entry_price = float(g.get("current_price", 0) or 0)
    except (TypeError, ValueError):
        entry_price = 0.0

    if entry_price <= 0:
        return

    clv_key = make_clv_key(slug, outcome, wallet)

    if clv_key not in clv_tracker:
        clv_tracker[clv_key] = {
            "slug": slug,
            "outcome": outcome,
            "wallet": wallet,
            "market": g.get("title", ""),
            "entry_price": entry_price,
            "first_seen_ts": now_ts,
            "last_seen_ts": now_ts,
            "latest_price": entry_price,
            "snapshot_clv": 0.0,
            "snapshot_clv_ready": False,
            "snapshot_clv_positive": False,
        }
    else:
        clv_tracker[clv_key]["last_seen_ts"] = now_ts
```

```
def update_clv_tracker(clv_tracker, scored_candidates, now_ts):
    if not isinstance(clv_tracker, dict):
        return {
            "tracked": 0,
            "ready": 0,
            "positive": 0,
            "avg_snapshot_clv_cents": 0.0,
        }

    current_price_lookup = {}
    for g in scored_candidates:
        if not isinstance(g, dict):
            continue

        slug = str(g.get("slug", "") or "").strip()
        outcome = str(g.get("outcome", "") or "").strip()
        wallet = str(g.get("wallet", "") or "").strip().lower()

        if not slug or not outcome or not wallet:
            continue

        try:
            current_price = float(g.get("current_price", 0) or 0)
        except (TypeError, ValueError):
            current_price = 0.0

        if current_price <= 0:
            continue

        current_price_lookup[make_clv_key(slug, outcome, wallet)] = current_price

    tracked = 0
    ready = 0
    positive = 0
    snapshot_clv_values = []

    for clv_key, row in clv_tracker.items():
        if not isinstance(row, dict):
            continue

        tracked += 1

        entry_price = float(row.get("entry_price", 0) or 0)
```

```
        first_seen_ts = int(row.get("first_seen_ts", 0) or 0)
        age_seconds = max(0, now_ts - first_seen_ts)

        if clv_key in current_price_lookup:
            latest_price = current_price_lookup[clv_key]
            row["latest_price"] = latest_price
            row["last_seen_ts"] = now_ts
        else:
            latest_price = float(row.get("latest_price", 0) or 0)

        snapshot_clv = latest_price - entry_price
        row["snapshot_clv"] = round(snapshot_clv, 4)

        is_ready = (
            age_seconds >= SNAPSHOT_CLV_MIN_AGE_SECONDS
            and age_seconds <= SNAPSHOT_CLV_MAX_AGE_SECONDS
            and latest_price > 0
            and entry_price > 0
        )

        row["snapshot_clv_ready"] = is_ready
        row["snapshot_clv_positive"] = bool(is_ready and snapshot_clv > 0)

        if is_ready:
            ready += 1
            snapshot_clv_values.append(snapshot_clv)
            if snapshot_clv > 0:
                positive += 1

    avg_snapshot_clv_cents = 0.0
    if snapshot_clv_values:
        avg_snapshot_clv_cents = round(
            (sum(snapshot_clv_values) / len(snapshot_clv_values)) * 100,
            2,
        )

    return {
        "tracked": tracked,
        "ready": ready,
        "positive": positive,
        "avg_snapshot_clv_cents": avg_snapshot_clv_cents,
    }

def get_follower_count(g):
```

```
    if not isinstance(g, dict):
        return 0

    wallet_count_scored = g.get("wallet_count_scored")
    if wallet_count_scored is not None:
        try:
            return max(0, int(wallet_count_scored) - 1)
        except (TypeError, ValueError):
            pass

    wallet_count = g.get("wallet_count")
    if wallet_count is not None:
        try:
            return max(0, int(wallet_count) - 1)
        except (TypeError, ValueError):
            pass

    confirmation_wallets = g.get("consensus_wallets")
    if confirmation_wallets is not None:
        try:
            return max(0, int(confirmation_wallets))
        except (TypeError, ValueError):
            pass

    return 0


def get_normalized_outcome_key(g):
    if not isinstance(g, dict):
        return ""
    return str(g.get("outcome", "") or "").strip().lower()


def get_game_conflict_key(g):
    if not isinstance(g, dict):
        return ""

    market_text = str(
        g.get("market")
        or g.get("title")
        or g.get("question")
        or g.get("slug")
        or ""
    ).strip().lower()
```

```
    if not market_text:
        return ""

    normalized = market_text
    normalized = re.sub(r"^spread:\s*", "", normalized)
    normalized = re.sub(r"^moneyline:\s*", "", normalized)
    normalized = re.sub(r"\([^)]+\)", "", normalized)
    normalized = re.sub(r"\b-?\d+(\.\d+)?\b", "", normalized)
    normalized = re.sub(r"\s+", " ", normalized).strip()

    return normalized


def get_same_side_prior_alert(g, alerted_bets):
    if not isinstance(g, dict):
        return None

    alert_key = get_bet_alert_key(g)
    if not alert_key:
        return None

    prior = alerted_bets.get(alert_key)
    if not isinstance(prior, dict):
        return None

    if str(prior.get("label", "") or "").upper() != "BET":
        return None

    return prior


def get_opposite_bet_alert(g, alerted_bets):
    if not isinstance(g, dict):
        return None

    slug = str(g.get("slug", "") or "").strip()
    outcome = str(g.get("outcome", "") or "").strip()
    if not slug or not outcome:
        return None

    current_game_conflict_key = get_game_conflict_key(g)
    current_outcome_key = get_normalized_outcome_key(g)
    opposite_priors = []
```

```
for prior_key, prior in alerted_bets.items():
    if not isinstance(prior, dict):
        continue

    if str(prior.get("label", "") or "").upper() != "BET":
        continue

    prior_slug = str(prior.get("slug", "") or "").strip()
    prior_outcome = str(prior.get("outcome", "") or "").strip()

    if not prior_slug or not prior_outcome:
        prior_key_str = str(prior_key or "").strip()
        if "||" in prior_key_str:
            parts = prior_key_str.split("||", 1)
            prior_slug = str(parts[0] or "").strip()
            prior_outcome = str(parts[1] or "").strip()

    if not prior_slug or not prior_outcome:
        continue

    if prior_slug == slug and prior_outcome == outcome:
        continue

    prior_game_conflict_key = str(prior.get("game_conflict_key", "") or "").strip().lower()
    prior_outcome_key = str(prior.get("normalized_outcome_key", "") or "").strip().lower()

    same_exact_market = (prior_slug == slug)
    same_game_family = (
        current_game_conflict_key
        and prior_game_conflict_key
        and prior_game_conflict_key == current_game_conflict_key
    )
    opposite_outcome = (
        current_outcome_key
        and prior_outcome_key
        and prior_outcome_key != current_outcome_key
    )

    if not ((same_exact_market or same_game_family) and opposite_outcome):
        continue

    opposite_priors.append(prior)
```

```
    if not opposite_priors:
        return None

    opposite_priors.sort(
        key=lambda x: int(x.get("last_alert_ts", 0) or 0),
        reverse=True,
    )
    return opposite_priors[0]

def get_prior_duplicate_bet_alert(g, alerted_bets):
    if not isinstance(g, dict):
        return None

    current_exact_key = get_bet_alert_key(g)
    current_totals_family_key = get_totals_family_key(g)
    current_side_family_key = get_side_family_key(g)

    prior_matches = []

    for prior_key, prior in alerted_bets.items():
        if not isinstance(prior, dict):
            continue

        if str(prior.get("label", "") or "").upper() != "BET":
            continue

        prior_exact_key = str(prior_key or "").strip()

        prior_slug = str(prior.get("slug", "") or "").strip()
        prior_outcome = str(prior.get("outcome", "") or "").strip()

        if not prior_slug or not prior_outcome:
            continue

        prior_g = {
            "slug": prior_slug,
            "outcome": prior_outcome,
            "market": prior.get("market", ""),
            "title": prior.get("title", ""),
            "question": prior.get("market", ""),
        }

        prior_totals_family_key = get_totals_family_key(prior_g)
        prior_side_family_key = get_side_family_key(prior_g)
```

```
        same_exact_key = (
            current_exact_key
            and prior_exact_key
            and current_exact_key == prior_exact_key
        )

        same_totals_family = (
            current_totals_family_key is not None
            and prior_totals_family_key is not None
            and current_totals_family_key == prior_totals_family_key
        )

        same_side_family = (
            current_side_family_key is not None
            and prior_side_family_key is not None
            and current_side_family_key == prior_side_family_key
        )

        if not (same_exact_key or same_totals_family or same_side_family):
            continue

        prior_matches.append(prior)

    if not prior_matches:
        return None

    prior_matches.sort(
        key=lambda x: int(x.get("last_alert_ts", 0) or 0),
        reverse=True,
    )
    return prior_matches[0]

def get_possible_flip_reason(g, opposite_prior):
    if not isinstance(g, dict) or not isinstance(opposite_prior, dict):
        return None
    try:
        new_score = float(g.get("score", 0) or 0)
    except (TypeError, ValueError):
        new_score = 0.0
    try:
        old_score = float(opposite_prior.get("score", 0) or 0)
    except (TypeError, ValueError):
        old_score = 0.0
```

```
try:
    new_edge = float(g.get("edge_pct", 0) or 0)
except (TypeError, ValueError):
    new_edge = 0.0
try:
    old_edge = float(opposite_prior.get("edge_pct", 0) or 0)
except (TypeError, ValueError):
    old_edge = 0.0
try:
    new_followers = int(get_follower_count(g) or 0)
except (TypeError, ValueError):
    new_followers = 0
try:
    old_followers = int(opposite_prior.get("followers", 0) or 0)
except (TypeError, ValueError):
    old_followers = 0
try:
    new_support_score = float(g.get("consensus_score", 0) or 0)
except (TypeError, ValueError):
    new_support_score = 0.0
try:
    old_support_score = float(opposite_prior.get("consensus_score", 0) or 0)
except (TypeError, ValueError):
    old_support_score = 0.0
try:
    new_total_size = float(g.get("total_size", 0) or 0)
except (TypeError, ValueError):
    new_total_size = 0.0
try:
    old_total_size = float(opposite_prior.get("total_size", 0) or 0)
except (TypeError, ValueError):
    old_total_size = 0.0

stronger_score = new_score >= (old_score + 5)
better_edge = new_edge >= (old_edge + 1.0)
larger_stake = (
    old_total_size > 0
    and new_total_size >= (old_total_size * 1.5)
    and (new_total_size - old_total_size) >= 1000
)
stronger_consensus = new_support_score >= (old_support_score + 20)
new_followers_added = new_followers > old_followers

if not (stronger_score and better_edge):
```

```
        return None

    if larger_stake:
        return (
            f"larger stake (${new_total_size:,.0f} vs ${old_total_size:,.0f})"
        )

    if stronger_consensus:
        return (
            f"stronger consensus score ({int(new_support_score)} vs {int(old_support_score)})"
        )

    if new_followers_added:
        return (
            f"new follower support ({new_followers} vs {old_followers})"
        )

    return None

def passes_new_bet_soft_floors(g, wallet_profiles):
    if not isinstance(g, dict):
        return False, {
            "soft_fail_count": 999,
            "justification_count": 0,
            "soft_fail_reasons": ["invalid_bet"],
            "justification_reasons": [],
        }

    try:
        total_notional = float(g.get("total_notional", 0) or 0)
    except Exception:
        total_notional = 0.0

    try:
        size_ratio = float(g.get("size_ratio", 0) or 0)
    except Exception:
        size_ratio = 0.0

    wallet = str(g.get("wallet", "") or "").strip().lower()
    wallet_profile = {}
    if isinstance(wallet_profiles, dict):
        wallet_profile = wallet_profiles.get(wallet, {}) or {}

    leaderboard_roi = wallet_profile.get("leaderboard_roi")
```

```
    try:
        leaderboard_roi = float(leaderboard_roi) if leaderboard_roi is not None else None
    except Exception:
        leaderboard_roi = None

    try:
        followers = int(get_follower_count(g) or 0)
    except Exception:
        followers = 0

    try:
        consensus_score = int(g.get("consensus_score", 0) or 0)
    except Exception:
        consensus_score = 0

    try:
        score = int(g.get("score", 0) or 0)
    except Exception:
        score = 0

    soft_fail_reasons = []

    if total_notional < BET_ALERT_SOFT_MIN_TOTAL_NOTIONAL:
        soft_fail_reasons.append("low_notional")

    if size_ratio < BET_ALERT_SOFT_MIN_SIZE_RATIO:
        soft_fail_reasons.append("low_ratio")

    if leaderboard_roi is None or leaderboard_roi <
BET_ALERT_SOFT_MIN_LEADERBOARD_ROI:
        soft_fail_reasons.append("low_roi")

    justification_reasons = []

    if score >= BET_ALERT_SOFT_STRONG_SCORE:
        justification_reasons.append("strong_score")

    if total_notional >= BET_ALERT_SOFT_STRONG_TOTAL_NOTIONAL:
        justification_reasons.append("strong_notional")

    if size_ratio >= BET_ALERT_SOFT_STRONG_SIZE_RATIO:
        justification_reasons.append("strong_ratio")
```

```
    if leaderboard_roi is not None and leaderboard_roi >=
BET_ALERT_SOFT_STRONG_LEADERBOARD_ROI:
        justification_reasons.append("strong_roi")

    if followers >= BET_ALERT_SOFT_STRONG_FOLLOWERS:
        justification_reasons.append("followers")

    if consensus_score >= BET_ALERT_SOFT_STRONG_CONSENSUS_SCORE:
        justification_reasons.append("consensus")

    try:
        minutes_to_start = g.get("minutes_to_start")
        if minutes_to_start is None:
            event_start_time = g.get("event_start_time")
            if event_start_time:
                from datetime import datetime, timezone
                event_dt = datetime.fromisoformat(str(event_start_time).replace("Z", "+00:00"))
                now_dt = datetime.now(timezone.utc)
                minutes_to_start = int((event_dt - now_dt).total_seconds() / 60)
    except Exception:
        minutes_to_start = None

    if (
        minutes_to_start is not None
        and 0 <= int(minutes_to_start) <=
BET_ALERT_SOFT_STRONG_MAX_MINUTES_TO_START
    ):
        justification_reasons.append("close_to_start")

    # --- hard stop for weak low-conviction signals ---
    if size_ratio < 1.25 and followers == 0 and consensus_score == 0:
        return False, {
            "soft_fail_count": 999,
            "justification_count": len(justification_reasons),
            "soft_fail_reasons": ["low_ratio_no_confirmation"],
            "justification_reasons": justification_reasons,
        }

    # --- hard stop for low-ROI signals unless they have real support ---
    if leaderboard_roi is not None and leaderboard_roi < 0.01:
        low_roi_override = (
            followers >= 1
            or consensus_score >= 50
            or size_ratio >= 3.0
```

```
        or total_notional >= 10000
    )

    if not low_roi_override:
        return False, {
            "soft_fail_count": 999,
            "justification_count": len(justification_reasons),
            "soft_fail_reasons": ["low_roi_no_override"],
            "justification_reasons": justification_reasons,
        }

  soft_fail_count = len(soft_fail_reasons)
  justification_count = len(justification_reasons)

  if soft_fail_count == 0:
    return True, {
        "soft_fail_count": soft_fail_count,
        "justification_count": justification_count,
        "soft_fail_reasons": soft_fail_reasons,
        "justification_reasons": justification_reasons,
    }

  required_justifications = BET_ALERT_SOFT_REQUIRED_JUSTIFICATIONS_ONE_FAIL
  if soft_fail_count >= 2:
    required_justifications = BET_ALERT_SOFT_REQUIRED_JUSTIFICATIONS_MULTI_FAIL

  passes = justification_count >= required_justifications

  return passes, {
    "soft_fail_count": soft_fail_count,
    "justification_count": justification_count,
    "soft_fail_reasons": soft_fail_reasons,
    "justification_reasons": justification_reasons,
  }

def is_possible_flip(g, opposite_prior):
  return get_possible_flip_reason(g, opposite_prior) is not None

def annotate_opposite_side_conflict(g, alerted_bets):
  if not isinstance(g, dict):
    return g

  annotated = dict(g)
  same_side_prior = get_same_side_prior_alert(annotated, alerted_bets)
```

```
    opposite_prior = get_opposite_bet_alert(annotated, alerted_bets)

    annotated["same_side_prior_exists"] = same_side_prior is not None
    annotated["opposite_conflict"] = False
    annotated["possible_flip"] = False
    annotated["possible_flip_reason"] = None
    annotated["opposite_outcome"] = None
    annotated["opposite_wallet"] = None
    annotated["opposite_score"] = None
    annotated["opposite_edge_pct"] = None
    annotated["opposite_followers"] = 0

    if same_side_prior is not None:
        return annotated

    if not opposite_prior:
        return annotated

    annotated["opposite_conflict"] = True
    annotated["opposite_outcome"] = opposite_prior.get("outcome")
    annotated["opposite_wallet"] = opposite_prior.get("wallet")
    annotated["opposite_score"] = opposite_prior.get("score")
    annotated["opposite_edge_pct"] = opposite_prior.get("edge_pct")
    annotated["opposite_followers"] = opposite_prior.get("followers", 0)
    annotated["possible_flip_reason"] = get_possible_flip_reason(annotated, opposite_prior)
    annotated["possible_flip"] = annotated["possible_flip_reason"] is not None
    return annotated

def should_send_bet_alert(g, alerted_bets, now_ts):
    if not isinstance(g, dict):
        return False

    if str(g.get("label", "") or "").upper() != "BET":
        return False

    g["duplicate_reason"] = None

    if g.get("opposite_conflict") and not g.get("possible_flip"):
        return False

    if g.get("possible_flip"):
        return True

    prior = get_prior_duplicate_bet_alert(g, alerted_bets)
```

```
if prior is None:
    return True

last_alert_ts = int(prior.get("last_alert_ts", 0) or 0)
seconds_since_last_alert = max(0, int(now_ts) - last_alert_ts)

old_price = prior.get("current_price")
new_price = g.get("current_price")

old_stake_pct = int(prior.get("stake_pct", 0) or 0)
new_stake_pct = int(g.get("stake_pct", 0) or 0)

old_score = int(prior.get("score", 0) or 0)
new_score = int(g.get("score", 0) or 0)

old_edge_pct = float(prior.get("edge_pct", 0) or 0)
new_edge_pct = float(g.get("edge_pct", 0) or 0)

old_followers = int(prior.get("followers", 0) or 0)
new_followers = int(get_follower_count(g) or 0)

old_total_size = float(prior.get("total_size", 0) or 0)
new_total_size = float(g.get("total_size", 0) or 0)

old_consensus = str(prior.get("consensus_type", "") or "").lower()
new_consensus = str(g.get("consensus_type", "") or "").lower()

old_consensus_score = int(prior.get("consensus_score", 0) or 0)
new_consensus_score = int(g.get("consensus_score", 0) or 0)

price_improved = False
price_not_much_worse = True
price_diff_cents = None

if old_price is not None and new_price is not None:
    try:
        price_diff_cents = round((float(old_price) - float(new_price)) * 100, 2)
        price_improved = float(new_price) <= (
            float(old_price) - BET_ALERT_MIN_PRICE_IMPROVEMENT
        )
        price_not_much_worse = float(new_price) <= (
            float(old_price) + BET_ALERT_MAX_ADVERSE_PRICE_MOVE
        )
```

```
    except Exception:
        price_improved = False
        price_not_much_worse = True
        price_diff_cents = None

if not price_not_much_worse:
    return False

stake_improved = (
    new_stake_pct >= (old_stake_pct + BET_ALERT_MIN_STAKE_PCT_INCREASE)
)

edge_improved = new_edge_pct >= (old_edge_pct + 1.0)

score_improved = new_score >= (old_score + BET_ALERT_MIN_SCORE_IMPROVEMENT)

followers_improved = (
    new_followers >= (old_followers + BET_ALERT_MIN_FOLLOWER_INCREASE)
)

size_improved = False
if old_total_size > 0:
    size_improved = (
        new_total_size >= max(
            old_total_size + BET_ALERT_MIN_NEW_SHARP_STAKE,
            old_total_size * 1.25,
        )
    )
else:
    size_improved = new_total_size >= BET_ALERT_MIN_NEW_SHARP_STAKE

consensus_improved = (
    (old_consensus != "full" and new_consensus == "full")
    or (
        new_consensus_score >= (
            old_consensus_score + BET_ALERT_MIN_CONSENSUS_SCORE_IMPROVEMENT
        )
        and new_consensus_score >= 60
    )
)

actionable_reasons = []

if stake_improved:
```

```
        actionable_reasons.append(
            f"higher stake ({old_stake_pct}% -> {new_stake_pct}%)"
        )

    if price_improved and price_diff_cents is not None:
        actionable_reasons.append(
            f"better price ({price_diff_cents:+.2f}c)"
        )

    if consensus_improved:
        if old_consensus != "full" and new_consensus == "full":
            actionable_reasons.append("upgraded to full consensus")
        else:
            actionable_reasons.append(
                f"stronger consensus score ({new_consensus_score} vs {old_consensus_score})"
            )

    if size_improved:
        actionable_reasons.append(
            f"larger sharp size (${new_total_size:,.0f} vs ${old_total_size:,.0f})"
        )

    if followers_improved:
        actionable_reasons.append(
            f"more followers ({new_followers} vs {old_followers})"
        )

    if score_improved and edge_improved:
        actionable_reasons.append(
            f"stronger score/edge ({old_score}/{old_edge_pct:.2f}% ->
{new_score}/{new_edge_pct:.2f}%)"
        )

    if not actionable_reasons:
        return False

    if seconds_since_last_alert < BET_ALERT_COOLDOWN_SECONDS:
        high_conviction_update = (
            stake_improved
            or price_improved
            or consensus_improved
            or (size_improved and followers_improved)
            or (score_improved and edge_improved)
        )
```

```
        if not high_conviction_update:
            return False

    g["duplicate_reason"] = " | ".join(actionable_reasons)
    return True

def store_bet_alert(g, alerted_bets, now_ts):
    alert_key = get_bet_alert_key(g)
    alerted_bets[alert_key] = {
        "last_alert_ts": now_ts,
        "slug": g.get("slug", ""),
        "outcome": g.get("outcome", ""),
        "wallet": g.get("wallet", ""),
        "title": g.get("title", ""),
        "market": g.get("market", ""),
        "game_conflict_key": get_game_conflict_key(g),
        "normalized_outcome_key": get_normalized_outcome_key(g),
        "current_price": g.get("current_price"),
        "score": g.get("score", 0),
        "edge_pct": g.get("edge_pct", 0),
        "consensus_type": g.get("consensus_type", ""),
        "consensus_score": g.get("consensus_score", 0),
        "label": g.get("label", ""),
        "stake_pct": g.get("stake_pct", 0),
        "total_size": g.get("total_size", 0),
        "buy_count": g.get("buy_count", 0),
        "followers": get_follower_count(g),
    }

def resolve_same_market_bet_conflicts(scored_candidates):
    grouped = defaultdict(list)

    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        slug = str(g.get("slug", "") or "").strip()
        if not slug:
            continue
        grouped[slug].append(g)

    resolved = []

    for slug, candidates in grouped.items():
        bet_candidates = [
```

```
    g for g in candidates
    if str(g.get("label", "") or "").upper() == "BET"
]

if len(bet_candidates) <= 1:
    resolved.extend(candidates)
    continue

def bet_rank(g):
    try:
        score = float(g.get("score", 0) or 0)
    except Exception:
        score = 0.0

    try:
        edge_pct = float(g.get("edge_pct", 0) or 0)
    except Exception:
        edge_pct = 0.0

    try:
        market_movement_abs = abs(float(g.get("market_movement_cents", 999) or 999))
    except Exception:
        market_movement_abs = 999.0

    try:
        size_ratio = float(g.get("size_ratio", 0) or 0)
    except Exception:
        size_ratio = 0.0

    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0

    return (
        score,
        edge_pct,
        -market_movement_abs,
        size_ratio,
        total_size,
    )

winning_bet = max(bet_candidates, key=bet_rank)
```

```
        for g in candidates:
            if g is winning_bet:
                resolved.append(g)
                continue

            if str(g.get("label", "") or "").upper() == "BET":
                g = dict(g)
                old_reason = str(g.get("reason", "") or "")
                g["label"] = "PASS"
                g["score"] = 0
                g["stake_pct"] = 0
                g["reason"] = (
                    f"{old_reason} | Rejected by same-market BET conflict "
                    f"(kept outcome={winning_bet.get('outcome', '')})"
                )

            resolved.append(g)

    return resolved

def resolve_same_market_bet_conflicts(scored_candidates):
    grouped = defaultdict(list)

    for g in scored_candidates:
        if not isinstance(g, dict):
            continue

        slug = str(g.get("slug", "") or "").strip()
        if not slug:
            continue

        grouped[slug].append(g)

    resolved = []

    for slug, candidates in grouped.items():
        bet_candidates = [
            g for g in candidates
            if str(g.get("label", "") or "").upper() == "BET"
        ]

        if len(bet_candidates) <= 1:
            resolved.extend(candidates)
            continue
```

```
def bet_rank(g):
    try:
        score = float(g.get("score", 0) or 0)
    except Exception:
        score = 0.0

    try:
        edge_pct = float(g.get("edge_pct", 0) or 0)
    except Exception:
        edge_pct = 0.0

    try:
        market_movement_abs = abs(float(g.get("market_movement_cents", 999) or 999))
    except Exception:
        market_movement_abs = 999.0

    try:
        size_ratio = float(g.get("size_ratio", 0) or 0)
    except Exception:
        size_ratio = 0.0

    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0

    return (
        score,
        edge_pct,
        -market_movement_abs,
        size_ratio,
        total_size,
    )

winning_bet = max(bet_candidates, key=bet_rank)

for g in candidates:
    if g is winning_bet:
        resolved.append(g)
        continue

    if str(g.get("label", "") or "").upper() == "BET":
        g = dict(g)
```

```
            old_reason = str(g.get("reason", "") or "")
            g["label"] = "PASS"
            g["score"] = 0
            g["stake_pct"] = 0
            g["reason"] = (
                f"{old_reason} | Rejected by same-market BET conflict "
                f"(kept outcome={winning_bet.get('outcome', '')})"
            )

        resolved.append(g)

    return resolved


def get_totals_family_key(g):
    if not isinstance(g, dict):
        return None

    market_text = str(
        g.get("market")
        or g.get("question")
        or g.get("title")
        or ""
    ).strip()

    outcome_text = str(g.get("outcome", "") or "").strip().lower()
    slug_text = str(g.get("slug", "") or "").strip().lower()

    market_text_lower = market_text.lower()

    is_total_market = (
        "o/u" in market_text_lower
        or "total" in market_text_lower
        or "-total-" in slug_text
        or "totals" in slug_text
    )

    if not is_total_market:
        return None

    if outcome_text not in {"over", "under"}:
        return None

    normalized_market = market_text_lower
```

```
    normalized_market = re.sub(r"o/u\s*\d+(\.\d+)?", "o/u", normalized_market)
    normalized_market = re.sub(r"\b\d+(\.\d+)?\b", "", normalized_market)
    normalized_market = re.sub(r"\s+", " ", normalized_market).strip()

    if not normalized_market:
        return None

    return ("totals_family", normalized_market, outcome_text)


def resolve_totals_family_bet_conflicts(scored_candidates):
    grouped = defaultdict(list)

    for g in scored_candidates:
        if not isinstance(g, dict):
            continue

        family_key = get_totals_family_key(g)
        if family_key is None:
            continue

        grouped[family_key].append(g)

    if not grouped:
        return list(scored_candidates)

    winning_ids = set()

    def bet_rank(g):
        try:
            stake_pct = float(g.get("stake_pct", 0) or 0)
        except Exception:
            stake_pct = 0.0

        try:
            score = float(g.get("score", 0) or 0)
        except Exception:
            score = 0.0

        try:
            edge_pct = float(g.get("edge_pct", 0) or 0)
        except Exception:
            edge_pct = 0.0
```

```
    try:
        market_movement_abs = abs(float(g.get("market_movement_cents", 999) or 999))
    except Exception:
        market_movement_abs = 999.0

    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0

    return (
        stake_pct,
        score,
        edge_pct,
        -market_movement_abs,
        total_size,
    )

for family_key, candidates in grouped.items():
    bet_candidates = [
        g for g in candidates
        if str(g.get("label", "") or "").upper() == "BET"
    ]

    if len(bet_candidates) <= 1:
        continue

    winning_bet = max(bet_candidates, key=bet_rank)
    winning_ids.add(id(winning_bet))

resolved = []

for g in scored_candidates:
    if not isinstance(g, dict):
        resolved.append(g)
        continue

    family_key = get_totals_family_key(g)
    if family_key is None:
        resolved.append(g)
        continue

    if id(g) in winning_ids:
```

```
            resolved.append(g)
            continue

        if str(g.get("label", "") or "").upper() == "BET":
            family_candidates = grouped.get(family_key, [])
            family_bets = [
                candidate for candidate in family_candidates
                if str(candidate.get("label", "") or "").upper() == "BET"
            ]

            if len(family_bets) > 1:
                winning_bet = max(family_bets, key=bet_rank)

                g = dict(g)
                old_reason = str(g.get("reason", "") or "")
                g["label"] = "PASS"
                g["score"] = 0
                g["stake_pct"] = 0
                g["reason"] = (
                    f"{old_reason} | Rejected by totals-family BET conflict "
                    f"(kept market={winning_bet.get('market', '')}, "
                    f"outcome={winning_bet.get('outcome', '')})"
                )

        resolved.append(g)

    return resolved


def get_side_family_key(g):
    if not isinstance(g, dict):
        return None

    market_text = str(
        g.get("market")
        or g.get("question")
        or g.get("title")
        or ""
    ).strip()

    outcome_text = str(g.get("outcome", "") or "").strip().lower()
    slug_text = str(g.get("slug", "") or "").strip().lower()

    market_text_lower = market_text.lower()
```

```
    is_side_market = (
        "spread:" in market_text_lower
        or "moneyline" in market_text_lower
        or "-spread-" in slug_text
        or "-winner" in slug_text
        or "-ml-" in slug_text
    )

    if not is_side_market:
        return None

    side_key = outcome_text
    if not side_key:
        return None

    normalized_market = market_text_lower

    normalized_market = re.sub(r"spread:\s*", "", normalized_market)
    normalized_market = re.sub(r"\([^)]+\)", "", normalized_market)
    normalized_market = re.sub(r"\b\d+(\.\d+)?\b", "", normalized_market)
    normalized_market = re.sub(r"\s+", " ", normalized_market).strip()

    if not normalized_market:
        return None

    return ("side_family", normalized_market, side_key)


def resolve_side_family_bet_conflicts(scored_candidates):
    grouped = defaultdict(list)

    for g in scored_candidates:
        if not isinstance(g, dict):
            continue
        family_key = get_side_family_key(g)
        if family_key is None:
            continue
        grouped[family_key].append(g)

    if not grouped:
        return list(scored_candidates)

    winning_ids = set()
```

```
def bet_rank(g):
    try:
        stake_pct = float(g.get("stake_pct", 0) or 0)
    except Exception:
        stake_pct = 0.0
    try:
        score = float(g.get("score", 0) or 0)
    except Exception:
        score = 0.0
    try:
        edge_pct = float(g.get("edge_pct", 0) or 0)
    except Exception:
        edge_pct = 0.0
    try:
        market_movement_abs = abs(float(g.get("market_movement_cents", 999) or 999))
    except Exception:
        market_movement_abs = 999.0
    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0

    return (
        stake_pct,
        score,
        edge_pct,
        -market_movement_abs,
        total_size,
    )

for family_key, candidates in grouped.items():
    bet_candidates = [
        g for g in candidates
        if str(g.get("label", "") or "").upper() == "BET"
    ]
    if len(bet_candidates) <= 1:
        continue

    winning_bet = max(bet_candidates, key=bet_rank)
    winning_ids.add(id(winning_bet))

resolved = []
```

```
    for g in scored_candidates:
        if not isinstance(g, dict):
            resolved.append(g)
            continue

        family_key = get_side_family_key(g)
        if family_key is None:
            resolved.append(g)
            continue

        if id(g) in winning_ids:
            resolved.append(g)
            continue

        if str(g.get("label", "") or "").upper() == "BET":
            family_candidates = grouped.get(family_key, [])
            family_bets = [
                candidate for candidate in family_candidates
                if str(candidate.get("label", "") or "").upper() == "BET"
            ]
            if len(family_bets) > 1:
                winning_bet = max(family_bets, key=bet_rank)
                g = dict(g)
                old_reason = str(g.get("reason", "") or "")
                g["label"] = "PASS"
                g["score"] = 0
                g["stake_pct"] = 0
                g["reason"] = (
                    f"{old_reason} | Rejected by side-family BET conflict "
                    f"(kept market={winning_bet.get('market', '')}, "
                    f"outcome={winning_bet.get('outcome', '')})"
                )

        resolved.append(g)

    return resolved

def dedupe_bet_candidates_for_cycle(bet_candidates):
    grouped = defaultdict(list)

    for g in bet_candidates:
        if not isinstance(g, dict):
            continue
```

```
    alert_key = get_bet_alert_key(g)
    if not alert_key:
        continue

    grouped[alert_key].append(g)

if not grouped:
    return []

def cycle_bet_rank(g):
    try:
        stake_pct = float(g.get("stake_pct", 0) or 0)
    except Exception:
        stake_pct = 0.0
    try:
        score = float(g.get("score", 0) or 0)
    except Exception:
        score = 0.0
    try:
        edge_pct = float(g.get("edge_pct", 0) or 0)
    except Exception:
        edge_pct = 0.0
    try:
        followers = float(get_follower_count(g) or 0)
    except Exception:
        followers = 0.0
    try:
        consensus_score = float(g.get("consensus_score", 0) or 0)
    except Exception:
        consensus_score = 0.0
    try:
        total_size = float(g.get("total_size", 0) or 0)
    except Exception:
        total_size = 0.0
    try:
        market_movement_abs = abs(float(g.get("market_movement_cents", 999) or 999))
    except Exception:
        market_movement_abs = 999.0
    try:
        seconds_since_last_buy = int(g.get("seconds_since_last_buy", 999999) or 999999)
    except Exception:
        seconds_since_last_buy = 999999

    return (
```

```
            stake_pct,
            score,
            edge_pct,
            followers,
            consensus_score,
            total_size,
            -market_movement_abs,
            -seconds_since_last_buy,
        )

    deduped = []

    for _, candidates in grouped.items():
        winning_bet = max(candidates, key=cycle_bet_rank)
        deduped.append(winning_bet)

    deduped.sort(key=cycle_bet_rank, reverse=True)
    return deduped

def classify_bet_alert_decision(g, alerted_bets, now_ts):
    if not isinstance(g, dict):
        return "skip_not_dict"

    if str(g.get("label", "") or "").upper() != "BET":
        return "skip_not_bet"

    if g.get("opposite_conflict") and not g.get("possible_flip"):
        return "skip_opposite_conflict"

    if g.get("possible_flip"):
        return "send_possible_flip"

    alert_key = get_bet_alert_key(g)
    prior = alerted_bets.get(alert_key)

    if prior is None:
        return "send_new_bet"

    old_price = prior.get("current_price")
    new_price = g.get("current_price")

    old_stake_pct = int(prior.get("stake_pct", 0) or 0)
    new_stake_pct = int(g.get("stake_pct", 0) or 0)
```

```
old_score = int(prior.get("score", 0) or 0)
new_score = int(g.get("score", 0) or 0)

old_edge_pct = float(prior.get("edge_pct", 0) or 0)
new_edge_pct = float(g.get("edge_pct", 0) or 0)

old_followers = int(prior.get("followers", 0) or 0)
new_followers = int(get_follower_count(g) or 0)

old_total_size = float(prior.get("total_size", 0) or 0)
new_total_size = float(g.get("total_size", 0) or 0)

old_consensus = str(prior.get("consensus_type", "") or "").lower()
new_consensus = str(g.get("consensus_type", "") or "").lower()

old_consensus_score = int(prior.get("consensus_score", 0) or 0)
new_consensus_score = int(g.get("consensus_score", 0) or 0)

price_improved = False
price_not_much_worse = True

if old_price is not None and new_price is not None:
    try:
        price_improved = float(new_price) <= (
            float(old_price) - BET_ALERT_MIN_PRICE_IMPROVEMENT
        )
        price_not_much_worse = float(new_price) <= (
            float(old_price) + BET_ALERT_MAX_ADVERSE_PRICE_MOVE
        )
    except Exception:
        price_improved = False
        price_not_much_worse = True

if not price_not_much_worse:
    return "skip_duplicate_price_worse"

stake_improved = (
    new_stake_pct >= (old_stake_pct + BET_ALERT_MIN_STAKE_PCT_INCREASE)
)

edge_improved = new_edge_pct >= (old_edge_pct + 1.0)

score_improved = new_score >= (old_score + BET_ALERT_MIN_SCORE_IMPROVEMENT)
```

```
followers_improved = (
    new_followers >= (old_followers + BET_ALERT_MIN_FOLLOWER_INCREASE)
)

size_improved = False
if old_total_size > 0:
    size_improved = (
        new_total_size >= max(
            old_total_size + BET_ALERT_MIN_NEW_SHARP_STAKE,
            old_total_size * 1.25,
        )
    )
else:
    size_improved = new_total_size >= BET_ALERT_MIN_NEW_SHARP_STAKE

consensus_improved = (
    (old_consensus != "full" and new_consensus == "full")
    or (
        new_consensus_score >= (
            old_consensus_score + BET_ALERT_MIN_CONSENSUS_SCORE_IMPROVEMENT
        )
        and new_consensus_score >= 60
    )
)

actionable_duplicate = (
    stake_improved
    or price_improved
    or consensus_improved
    or size_improved
    or followers_improved
    or (score_improved and edge_improved)
)

if not actionable_duplicate:
    return "skip_duplicate_not_actionable"

last_alert_ts = int(prior.get("last_alert_ts", 0) or 0)
seconds_since_last_alert = max(0, int(now_ts) - last_alert_ts)

if seconds_since_last_alert < BET_ALERT_COOLDOWN_SECONDS:
    high_conviction_update = (
        stake_improved
        or price_improved
```

```
            or consensus_improved
            or (size_improved and followers_improved)
            or (score_improved and edge_improved)
        )
        if not high_conviction_update:
            return "skip_duplicate_cooldown"

    return "send_actionable_duplicate"

def get_bet_age_bucket(seconds_since_last_buy):
    try:
        age_seconds = int(seconds_since_last_buy)
    except Exception:
        return "unknown"

    if age_seconds <= 60:
        return "00-60s"
    if age_seconds <= 180:
        return "01-03m"
    if age_seconds <= 300:
        return "03-05m"
    if age_seconds <= 600:
        return "05-10m"
    if age_seconds <= 1200:
        return "10-20m"
    return "20m+"


def summarize_bet_age_buckets(bets):
    summary = {
        "count": 0,
        "avg_age_seconds": None,
        "median_age_seconds": None,
        "bucket_counts": {
            "00-60s": 0,
            "01-03m": 0,
            "03-05m": 0,
            "05-10m": 0,
            "10-20m": 0,
            "20m+": 0,
            "unknown": 0,
        },
    }
```

```
    if not isinstance(bets, list):
        return summary

    age_values = []

    for g in bets:
        if not isinstance(g, dict):
            continue

        raw_age = g.get("seconds_since_last_buy")
        bucket = get_bet_age_bucket(raw_age)
        summary["bucket_counts"][bucket] = summary["bucket_counts"].get(bucket, 0) + 1

        try:
            age_seconds = int(raw_age)
            if age_seconds >= 0:
                age_values.append(age_seconds)
        except Exception:
            pass

    summary["count"] = len(age_values)

    if age_values:
        age_values = sorted(age_values)
        summary["avg_age_seconds"] = round(sum(age_values) / len(age_values), 1)

        n = len(age_values)
        if n % 2 == 1:
            summary["median_age_seconds"] = age_values[n // 2]
        else:
            summary["median_age_seconds"] = round(
                (age_values[n // 2 - 1] + age_values[n // 2]) / 2,
                1,
            )

    return summary

def summarize_numeric_distribution(values, buckets):
    summary = {
        "count": 0,
        "avg": None,
        "median": None,
        "bucket_counts": {label: 0 for label in buckets},
    }
```

```
    clean_values = []

    for v in values:
        try:
            val = float(v)
            clean_values.append(val)
        except Exception:
            continue

    summary["count"] = len(clean_values)

    if not clean_values:
        return summary

    clean_values.sort()

    summary["avg"] = round(sum(clean_values) / len(clean_values), 2)

    n = len(clean_values)
    if n % 2 == 1:
        summary["median"] = clean_values[n // 2]
    else:
        summary["median"] = round(
            (clean_values[n // 2 - 1] + clean_values[n // 2]) / 2,
            2
        )

    for val in clean_values:
        for label, condition in buckets.items():
            if condition(val):
                summary["bucket_counts"][label] += 1
                break

    return summary

def get_minutes_to_start(event_start_time, now_ts):
    if not event_start_time:
        return None

    try:
        from datetime import datetime, timezone

        event_dt = datetime.fromisoformat(str(event_start_time).replace("Z", "+00:00"))
```

```
        now_dt = datetime.fromtimestamp(int(now_ts), tz=timezone.utc)
        minutes_to_start = int((event_dt - now_dt).total_seconds() / 60)
        return minutes_to_start
    except Exception:
        return None


def get_time_to_start_bucket(minutes_to_start):
    if minutes_to_start is None:
        return "unknown"

    try:
        m = int(minutes_to_start)
    except Exception:
        return "unknown"

    if m < 0:
        return "live_or_started"
    if m <= 10:
        return "00-10m"
    if m <= 30:
        return "10-30m"
    if m <= 60:
        return "30-60m"
    if m <= 180:
        return "01-03h"
    if m <= 360:
        return "03-06h"
    if m <= 720:
        return "06-12h"
    return "12h+"

def run_pipeline(wallet_profiles):
    global TRACKED_WALLETS

    all_trades = []

    for wallet in TRACKED_WALLETS:
        try:
            wallet_trades = load_activity(wallet)
            all_trades.extend(wallet_trades)
        except Exception as e:
            print(f"[Wallet fetch error] {wallet} -> {repr(e)}")
```

```
recent_trades, cutoff_ts, now_ts = filter_recent_trades(
    all_trades,
    hours_lookback=HOURS_LOOKBACK
)
valid_buy_trades = filter_valid_buy_trades(recent_trades)

wallet_medians = compute_wallet_medians(valid_buy_trades)

accumulation_groups = group_accumulation_candidates(valid_buy_trades)
accumulation_groups = mark_recent_paired_activity(accumulation_groups)
accumulation_groups = apply_cross_wallet_sequence_roles(accumulation_groups)

real_candidates = []
filtered_counts = {
    "buy_count": 0,
    "paired_recent": 0,
    "price_range": 0,
}

for g in accumulation_groups:
    if not isinstance(g, dict):
        continue

    # allow single large trades for leader/early wallets
    if g["buy_count"] < 2:
        if str(g.get("sequence_role", "")).lower() not in {"leader", "early"}:
            filtered_counts["buy_count"] += 1
            continue

    if g["paired_recent"]:
        filtered_counts["paired_recent"] += 1
        continue

    if not is_actionable_accumulation_group(g):
        filtered_counts["price_range"] += 1
        continue

    real_candidates.append(g)

positions = []

for wallet in TRACKED_WALLETS:
    try:
        wallet_positions = load_positions(wallet)
```

```
        positions.extend(wallet_positions)
    except Exception as e:
        print(f"[Position fetch error] {wallet} -> {repr(e)}")

position_lookup = build_position_lookup(positions)
fair_price_lookup = build_fair_price_lookup(accumulation_groups)
wallet_profiles = compute_dynamic_wallet_weights(wallet_profiles)

scored_candidates = attach_position_data_and_score(
    real_candidates,
    position_lookup,
    wallet_medians,
    wallet_profiles,
    fair_price_lookup
)

consensus_list = build_cross_wallet_consensus(
    accumulation_groups,
    scored_candidates,
    wallet_profiles
)

scored_candidates = apply_consensus_upgrades(
    scored_candidates,
    consensus_list,
    wallet_profiles
)

scored_candidates = resolve_same_market_bet_conflicts(scored_candidates)
scored_candidates = resolve_totals_family_bet_conflicts(scored_candidates)
scored_candidates = resolve_side_family_bet_conflicts(scored_candidates)

wallet_profiles = update_wallet_profiles(
    wallet_profiles,
    accumulation_groups,
    scored_candidates
)

wallet_profiles = compute_dynamic_wallet_weights(wallet_profiles)

active_wallets = filter_active_wallets(wallet_profiles)
gated_wallets = apply_wallet_stability_gating(wallet_profiles, active_wallets)

if gated_wallets:
```

```
        TRACKED_WALLETS = gated_wallets

    consensus_diagnostics = build_consensus_diagnostics(
        accumulation_groups,
        scored_candidates
    )

    return {
        "all_trades": all_trades,
        "recent_trades": recent_trades,
        "valid_buy_trades": valid_buy_trades,
        "accumulation_groups": accumulation_groups,
        "filtered_counts": filtered_counts,
        "consensus_list": consensus_list,
        "consensus_diagnostics": consensus_diagnostics,
        "wallet_profiles": wallet_profiles,
        "scored_candidates": scored_candidates,
        "now_ts": now_ts,
        "cutoff_ts": cutoff_ts,
        "active_wallet_count": len(TRACKED_WALLETS),
        "active_wallet_failure_counts": dict(ACTIVE_WALLET_FAILURE_COUNTS),
    }

def price_to_american_odds(price):
    try:
        p = float(price)
        if not (0 < p < 1):
            return None
        if p >= 0.5:
            odds = -(p / (1 - p)) * 100
        else:
            odds = ((1 - p) / p) * 100
        return int(round(odds))
    except Exception:
        return None

def format_event_start(event_start):
    if not event_start:
        return None, None

    try:
        from datetime import datetime, timezone
        import pytz
```

```
        dt = datetime.fromisoformat(event_start.replace("Z", "+00:00"))

        local_tz = pytz.timezone("America/Chicago")
        local_dt = dt.astimezone(local_tz)

        now = datetime.now(timezone.utc)
        minutes_to_start = int((dt - now).total_seconds() / 60)

        formatted_time = local_dt.strftime("%m/%d %-I:%M %p")

        return formatted_time, minutes_to_start

    except Exception:
        return None, None

def is_live_market(g):
    market_text = str(g.get("market") or "").lower()
    question_text = str(g.get("question") or "").lower()
    title_text = str(g.get("title") or "").lower()
    slug_text = str(g.get("slug") or "").lower()

    combined = f"{market_text} | {question_text} | {title_text} | {slug_text}"

    live_markers = [
        " live ",
        "live:",
        "(live",
        "[live",
        " in-play",
        "in play",
        "currently winning",
        "currently leading",
        "right now",
        "-live-",
        "_live_",
        "-in-play-",
        "-inplay-",
    ]

    return any(marker in combined for marker in live_markers)

def send_pushover_bet_alert(g):
    if not PUSHOVER_ENABLED:
        return
```

```
    market_text = str(g.get("market") or g.get("question") or g.get("title") or g.get("slug") or
"").strip()
    outcome_text = str(g.get("outcome", "") or "").strip()
    stake_pct = g.get("stake_pct")
    current_price = g.get("current_price")
    entry_price = g.get("wallet_entry_price")
    edge_pct = g.get("edge_pct")
    last_secs = g.get("seconds_since_last_buy")
    total_size_bought = g.get("total_size")
    size_ratio = g.get("size_ratio")

    current_price_str = "N/A"
    entry_price_str = "N/A"

    try:
        p = float(current_price)
        if 0 < p < 1:
            if p >= 0.5:
                odds = -(p / (1 - p)) * 100
            else:
                odds = ((1 - p) / p) * 100
            odds = int(round(odds))
            current_price_str = f"+{odds}" if odds > 0 else f"{odds}"
    except Exception:
        pass

    try:
        ep = float(entry_price)
        if 0 < ep < 1:
            if ep >= 0.5:
                entry_odds = -(ep / (1 - ep)) * 100
            else:
                entry_odds = ((1 - ep) / ep) * 100
            entry_odds = int(round(entry_odds))
            entry_price_str = f"+{entry_odds}" if entry_odds > 0 else f"{entry_odds}"
    except Exception:
        pass
    title = "BET ALERT"
    if g.get("duplicate_reason"):
        title = f"DUPLICATE BET – {g['duplicate_reason']}"
    elif g.get("possible_flip"):
        flip_reason = str(g.get("possible_flip_reason") or "").strip()
        if flip_reason:
```

```
        title = f"POSSIBLE FLIP – {flip_reason}"
    else:
        title = "POSSIBLE FLIP"
elif g.get("opposite_conflict"):
    title = "OPPOSITE-SIDE CONFLICT"

event_start = g.get("event_start_time")
formatted_start, minutes_to_start = format_event_start(event_start)

start_str = "Unknown"
if formatted_start:
    if minutes_to_start is not None:
        if minutes_to_start >= 0:
            start_str = f"{formatted_start} ({minutes_to_start}m)"
        else:
            start_str = f"{formatted_start} (live)"
    else:
        start_str = formatted_start

market_phase = str(g.get("market_phase") or "").strip()
if not market_phase:
    if minutes_to_start is not None and minutes_to_start < 0:
        market_phase = "Live"
    else:
        market_phase = "Pre-Game"

total_size_str = "N/A"
try:
    total_size_str = f"{float(total_size_bought):,.2f}"
except Exception:
    pass

consensus_total_size = g.get("consensus_total_size_scored")
if consensus_total_size is None:
    consensus_total_size = g.get("consensus_total_size")
if consensus_total_size is None:
    consensus_total_size = g.get("total_size_scored")
if consensus_total_size is None:
    consensus_total_size = g.get("total_size_all")

consensus_total_size_str = "N/A"
try:
    consensus_total_size_str = f"{float(consensus_total_size):,.2f}"
except Exception:
```

```
    pass

size_ratio_str = "N/A"
try:
    size_ratio_str = f"{float(size_ratio):.2f}"
except Exception:
    pass

last_bet_str = "N/A"
try:
    last_bet_str = f"{int(last_secs)}s"
except Exception:
    pass

consensus_wallet_count = g.get("consensus_wallet_count")
if consensus_wallet_count is None:
    consensus_wallet_count = g.get("wallet_count_scored")
if consensus_wallet_count is None:
    consensus_wallet_count = g.get("wallet_count")

consensus_wallets_str = "N/A"
try:
    consensus_wallets_str = str(int(consensus_wallet_count))
except Exception:
    pass

consensus_score = g.get("consensus_score")
consensus_score_str = "N/A"
try:
    consensus_score_str = str(int(float(consensus_score)))
except Exception:
    pass

leader_wallet = g.get("wallet")

leader_roi = None
if leader_wallet and wallet_profiles and leader_wallet in wallet_profiles:
    leader_roi = wallet_profiles.get(leader_wallet, {}).get("roi")

    if leader_roi is None:
        leader_roi_str = "N/A"
    else:
        try:
            leader_roi_str = f"{leader_roi:+.1f}%"
```

```
        except:
            leader_roi_str = "N/A"

consensus_wallets_value = g.get("consensus_wallets")
consensus_total_stake_value = g.get("consensus_total_stake")

if consensus_wallets_value in (None, "", "N/A"):
    total_wallets_display = 1
else:
    try:
        total_wallets_display = max(1, int(consensus_wallets_value) + 1)
    except (TypeError, ValueError):
        total_wallets_display = 1

leader_wallet_stake = g.get("total_size_bought")

try:
    leader_wallet_stake = float(leader_wallet_stake)
except (TypeError, ValueError):
    leader_wallet_stake = 0.0

total_stake_display = leader_wallet_stake

if consensus_total_stake_value not in (None, "", "N/A"):
    try:
        total_stake_display += float(consensus_total_stake_value)
    except (TypeError, ValueError):
        total_stake_display = leader_wallet_stake

edge_pct_display = f"{float(edge_pct):+.2f}".rstrip("0").rstrip(".")
followers_display = get_follower_count(g)

try:
    leader_size_display = f"${int(round(float(total_size_str.replace(',', '')))):,}"
except (TypeError, ValueError, AttributeError):
    leader_size_display = f"${total_size_str}"

wallet_addr = str(g.get("wallet", "") or "").strip().lower()
leaderboard_roi = wallet_profiles.get(wallet_addr, {}).get("leaderboard_roi")

if isinstance(leaderboard_roi, (int, float)):
   leader_roi_display = f"{round(leaderboard_roi * 100, 1)}%"
else:
    leader_roi_display = "N/A"
```

```
market_text_lower = str(market_text).lower()
slug_lower = str(g.get("slug", "") or "").lower()

futures_text_markers = [
    " will ",
    " by ",
    "who will",
    "qualify",
    "enter iran",
    "win the",
    "playoffs",
    "reach the",
    "advance to",
    "earn promotion",
    "be relegated",
    "champion",
    "nominee",
    "elected",
    "qualify",
    "price"
]

sports_game_slug_markers = [
    "nba-",
    "nhl-",
    "mlb-",
    "nfl-",
    "wnba-",
    "epl-",
    "cbb-",
    "atp-",
    "wta-",
    "spread",
    "total",
]

is_futures_market = False

if "?" in str(market_text):
# --- Futures detection (refined) ---
    is_futures_market = False

# Strong futures keywords (non-game concepts)
```

```
    if any(marker in market_text_lower for marker in [
        "enter",
        "election",
        "nominee",
        "champion",
        "tournament",
        "to make",
        "advance to",
        "be relegated",
        "earn promotion",
        "qualify"
        "oil"
        "championship"
        "price of"
        "inflation"
    ]):
        is_futures_market = True

    # Long time horizon (very strong signal)
    elif "(live)" in str(start_str).lower() and "by " in market_text_lower:
        is_futures_market = True

    # Slug fallback for non-sports markets
    elif slug_lower and not any(marker in slug_lower for marker in sports_game_slug_markers):
        if any(token in slug_lower for token in ["iran", "election," "qualify," "playoffs," "advance,"
"champion"]):
            is_futures_market = True
    elif slug_lower and not any(marker in slug_lower for marker in sports_game_slug_markers):
        if any(token in slug_lower for token in ["will-", "by-", "qualify", "iran", "election"]):
            is_futures_market = True

    is_live = False

    try:
        if "live" in str(start_str).lower():
            is_live = True
    except:
        pass

    # Futures always overrides live/pre-game
    if is_futures_market:
        phase_label = "Futures/Other"
    elif is_live:
        phase_label = "Live"
```

```
    else:
        phase_label = "Pre-Game"

    conflict_line = ""
    if g.get("opposite_conflict"):
        conflict_status = "Possible flip" if g.get("possible_flip") else "Opposite-side conflict"
        conflict_line = (
            f"{conflict_status}: already alerted on {g.get('opposite_outcome', 'N/A')}\n"
        )

    score_display = "N/A"
    try:
        score_display = f"{int(round(float(g.get('score', 0) or 0)))}/100"
    except Exception:
        score_display = "N/A"

    message = (
        f"{phase_label} | {market_text}\n"
        f"Bet: {outcome_text} | Stake: {stake_pct}%\n"
        f"Score: {score_display} | Edge: {edge_pct_display}%\n"
        f"Leader Size: {leader_size_display} | Ratio: {size_ratio_str} | ROI: {leader_roi_display}\n"
        f"Current Price: {current_price_str} | Entry Price: {entry_price_str}\n"
        f"Followers: {followers_display}\n"
        f"Start: {start_str}\n"
        f"Last Bet Placed: {last_bet_str}\n"
        f"Wallet: {g.get('wallet', 'N/A')}"
    )

    try:
        api_token = str(PUSHOVER_API_TOKEN).strip()

        if isinstance(PUSHOVER_USER_KEYS, (list, tuple)):
            user_keys = [str(x).strip() for x in PUSHOVER_USER_KEYS if str(x).strip()]
        else:
            user_keys = [str(PUSHOVER_USER_KEYS).strip()] if
str(PUSHOVER_USER_KEYS).strip() else []

        if not user_keys:
            print("Pushover send skipped: no valid user keys configured")
            return

        for user_key in user_keys:
            try:
                print("Pushover debug - user key length:", len(user_key))
```

```
            print("Pushover debug - user key preview:", f"{user_key[:4]}...{user_key[-4:]}")

            validate_resp = requests.post(
                "https://api.pushover.net/1/users/validate.json",
                data={
                    "token": api_token,
                    "user": user_key,
                },
                timeout=10,
            )
            print(f"Pushover validate response ({user_key[:4]}...{user_key[-4:]}):",
validate_resp.status_code, validate_resp.text)

            if validate_resp.status_code != 200:
                continue

            validate_json = validate_resp.json()
            if validate_json.get("status") != 1:
                continue

            resp = requests.post(
                "https://api.pushover.net/1/messages.json",
                data={
                    "token": api_token,
                    "user": user_key,
                    "title": title,
                    "message": message,
                    "priority": PUSHOVER_PRIORITY,
                },
                timeout=10,
            )
            print(f"Pushover response ({user_key[:4]}...{user_key[-4:]}):", resp.status_code,
resp.text)

        except Exception as inner_e:
            print(f"Pushover send failed for {user_key[:4]}...{user_key[-4:]}: {inner_e}")

    except Exception as e:
        print(f"Pushover send failed: {e}")
def print_signal(g):
    print("-" * 80)
    username = g.get("username", "")
    wallet = g.get("wallet", "")
```

```
if username:
    print(f"Wallet:             {username} ({wallet})")
else:
    print(f"Wallet:             {wallet}")
print(f"Market:             {g['title']}")
print(f"Slug:               {g['slug']}")
print(f"Outcome:            {g['outcome']}")
print(f"Paired recent:      {g['paired_recent']}")
print(f"Sequence role:      {g.get('sequence_role', 'N/A')}")
print(f"Buy count:          {g['buy_count']}")
print(f"Total size bought:  {g['total_size']}")
print(f"Avg trade size:     {round(g['avg_trade_size'], 2)}")
print(f"Avg trade price:    {g['avg_trade_price']}")
print(f"Avg trade notional: {g.get('avg_trade_notional', 'N/A')}")
current_price = g.get("current_price")
american_odds = price_to_american_odds(current_price)

fair_price = g.get("fair_price")
fair_american_odds = g.get("fair_american_odds")
edge_pct = g.get("edge_pct")

market_phase = g.get("market_phase")
event_start = g.get("event_start_time")

print(f"Market phase:       {market_phase}")
print(f"Event start:        {event_start}")
print(f"Current price:      {current_price}")

if american_odds is not None:
    if american_odds > 0:
        odds_str = f"+{american_odds}"
    else:
        odds_str = f"{american_odds}"
    print(f"American odds:      {odds_str}")
else:
    odds_str = "N/A"
    print(f"American odds:      N/A")

print(f"Fair price:         {fair_price}")

if fair_american_odds is not None:
    if fair_american_odds > 0:
        fair_odds_str = f"+{fair_american_odds}"
    else:
```

```
        fair_odds_str = f"{fair_american_odds}"
    print(f"Fair odds:          {fair_odds_str}")
else:
    fair_odds_str = "N/A"
    print(f"Fair odds:          N/A")

print(f"Edge %:             {edge_pct}")
print(f"Wallet entry price:  {g.get('wallet_entry_price')}")
print(f"Market movement:     {g.get('market_movement_cents')} cents")
print(f"Time span (seconds): {g.get('seconds_span', 'N/A')}")
print(f"Since last buy (s):  {g.get('seconds_since_last_buy', 'N/A')}")
import datetime

last_ts = g.get("last_timestamp")
if last_ts:
    try:
        dt = datetime.datetime.fromtimestamp(int(last_ts)).astimezone()
        formatted_time = dt.strftime("%Y-%m-%d %I:%M:%S %p %Z")
        print(f"Last bet time:      {formatted_time}")
    except Exception:
        pass
print(f"Accumulation points: {g['accumulation_points']}")
print(f"Size ratio:         {g['size_ratio']}")
print(f"Size points:        {g['size_points']}")
print(f"Support tier:       {g.get('consensus_type', 'None')}")
print(f"Support score:      {g.get('consensus_score', 0)}")
print(f"Support weight:     {g.get('consensus_weighted', 0)}")
print(f"Support boost:      {g.get('consensus_upgrade', False)}")
print(f"Followers:          {g.get('followers', 0)}")
print(f"Label:              {g['label']}")
print(f"Score:              {g['score']}")
print(f"Stake %:            {g['stake_pct']}")
print(f"Reason:             {g['reason']}")
clv_key = make_clv_key(g.get("slug", ""), g.get("outcome", ""), g.get("wallet", ""))
clv_row = clv_tracker.get(clv_key, {}) if isinstance(clv_tracker, dict) else {}
snapshot_clv = clv_row.get("snapshot_clv")
snapshot_clv_ready = clv_row.get("snapshot_clv_ready", False)
latest_price = clv_row.get("latest_price")
clv_status = "Not tracked yet"
if clv_row:
    if snapshot_clv_ready and snapshot_clv is not None:
        try:
            snapshot_clv_cents = round(float(snapshot_clv) * 100, 2)
            clv_status = f"{snapshot_clv_cents:+.2f} cents"
```

```
        except Exception:
            clv_status = "Ready but unreadable"
    else:
        entry_price_for_clv = clv_row.get("entry_price")
        try:
            if entry_price_for_clv is not None and latest_price is not None:
                live_clv_cents = round((float(latest_price) - float(entry_price_for_clv)) * 100, 2)
                clv_status = f"Pending ({live_clv_cents:+.2f} cents so far)"
            else:
                clv_status = "Pending"
        except Exception:
            clv_status = "Pending"
print(f"Snapshot CLV:       {clv_status}")
market_text = g.get("market")
if not market_text:
    market_text = g.get("question")
if not market_text:
    market_text = g.get("title")
if not market_text:
    market_text = g.get("slug")
market_text = str(market_text or "").strip()

outcome_text = str(g.get("outcome", "") or "").strip()

current_price = g.get("current_price")
stake_pct = g.get("stake_pct")

odds_str = "N/A"
try:
    p = float(current_price)
    if 0 < p < 1:
        if p >= 0.5:
            odds = - (p / (1 - p)) * 100
        else:
            odds = ((1 - p) / p) * 100

        odds = int(round(odds))
        odds_str = f"+{odds}" if odds > 0 else f"{odds}"
except Exception:
    pass

edge_pct = g.get("edge_pct")
market_phase = g.get("market_phase")
event_start = g.get("event_start_time")
```

```
    movement = g.get("market_movement_cents")
    print(f"Summary: {market_phase} | {market_text} | {outcome_text} | {odds_str} | move
{movement}c | {stake_pct}% | start {event_start}")
    print("-" * 80)

def print_consensus(consensus_list):
    print("CROSS-WALLET CONSENSUS")
    print("=" * 80)

    if not consensus_list:
        print("No cross-wallet consensus signals.")
        return

    for c in consensus_list[:10]:
        print("-" * 80)
        print(f"Market:               {c['market']}")
        print(f"Outcome:              {c['outcome']}")
        print(f"Consensus type:       {c['consensus_type']}")
        print(f"Wallet count (all):   {c['wallet_count_all']}")
        print(f"Wallet count (scored): {c['wallet_count_scored']}")
        print(f"Weighted all:         {c.get('weighted_wallet_score_all', 'N/A')}")
        print(f"Weighted scored:      {c.get('weighted_wallet_score_scored', 'N/A')}")
        print(f"Total size (all):     {round(c['total_size_all'], 2)}")
        print(f"Total size (scored):  {round(c['total_size_scored'], 2)}")
        print(f"Consensus score:      {c['consensus_score']}")

if __name__ == "__main__":
    TRACKED_WALLETS, leaderboard_rows = load_leaderboard_wallets(
        limit=LEADERBOARD_WALLET_LIMIT,
        offsets=LEADERBOARD_WALLET_OFFSETS,
    )

    if not TRACKED_WALLETS:
        raise RuntimeError("No leaderboard wallets loaded.")

    alerted_bets = load_alerted_bets()
    clv_tracker = load_clv_tracker()
    tracked_bets = load_tracked_bets()
    signal_metrics_history = load_signal_metrics_history()
    print(f"BASE_DIR: {BASE_DIR}")
    print(f"DATA_DIR in use: {DATA_DIR}")
    print(f"ALERTED_BETS_PATH: {ALERTED_BETS_PATH}")
    print(f"CLV_TRACKER_PATH: {CLV_TRACKER_PATH}")
```

```
print(f"TRACKED_BETS_PATH: {TRACKED_BETS_PATH}")
print(f"SIGNAL_METRICS_HISTORY_PATH: {SIGNAL_METRICS_HISTORY_PATH}")
print(f"Loaded alerted bets: {len(alerted_bets)}")
print(f"Loaded CLV tracker rows: {len(clv_tracker)}")
print(f"Loaded tracked bets: {len(tracked_bets)}")
print(f"Loaded signal metrics rows: {len(signal_metrics_history)}")
wallet_profiles = init_wallet_profiles(TRACKED_WALLETS)
enrich_wallet_profiles_with_leaderboard(wallet_profiles, leaderboard_rows)

# --- optional test alert ---
if PUSHOVER_TEST_ALERT:
    send_pushover_bet_alert({
        "market": "TEST ALERT",
        "outcome": "Test Bet",
        "stake_pct": 40,
        "current_price": 0.55,
        "seconds_since_last_buy": 5,
        "sequence_role": "leader",
    })

print(f"Loaded {len(TRACKED_WALLETS)} leaderboard wallets")
print("First 10 leaderboard wallets:")
for wallet in TRACKED_WALLETS[:10]:
    print(f"  {wallet}")
print(f"Starting signal monitor - polling every {POLL_SECONDS} seconds")
print("=" * 80)

while True:
    cycle_bet_alerts = []
    rejected_candidates = []

    try:
        result = run_pipeline(wallet_profiles)
        wallet_profiles = result["wallet_profiles"]

        print("=" * 80)
        print("ACTIVITY PIPELINE SUMMARY")
        print(f"All trades loaded:        {len(result['all_trades'])}")
        print(f"Recent trades kept:       {len(result['recent_trades'])}")
        print(f"Valid BUY trades:         {len(result['valid_buy_trades'])}")
        print(f"Accumulation groups:      {len(result['accumulation_groups'])}")
        print(f"Scored candidates:        {len(result['scored_candidates'])}")
        print(f"Filtered (buy_count):     {result['filtered_counts']['buy_count']}")
        print(f"Filtered (paired):        {result['filtered_counts']['paired_recent']}")
```

```
print(f"Filtered (price range):   {result['filtered_counts']['price_range']}")
print(f"Active wallets retained:  {result['active_wallet_count']}")
print(f"Failure trackers active:  {len(result['active_wallet_failure_counts'])}")
print(f"Lookback hours:           {HOURS_LOOKBACK}")
print(f"Current timestamp:        {result['now_ts']}")
print(f"Cutoff timestamp:         {result['cutoff_ts']}")
print("=" * 80)

print("WALLET PROFILE SUMMARY")
top_wallets = sorted(
    result["wallet_profiles"].values(),
    key=lambda x: (
        float(x.get("dynamic_weight", 1.0) or 1.0),
        int(x.get("clv_observations", 0) or 0),
        int(x.get("evaluated_clusters", 0) or 0),
    ),
    reverse=True,
)[:15]

for profile in top_wallets:
    print("-" * 80)
    print(f"Wallet:              {profile['wallet']}")
    print(f"Evaluated trades:    {profile['evaluated_trades']}")
    print(f"Evaluated clusters:  {profile['evaluated_clusters']}")
    print(f"CLV observations:    {profile['clv_observations']}")
    print(f"Avg forward CLV:     {profile['avg_forward_clv']}")
    print(f"Positive CLV rate:   {profile['positive_clv_rate']}")
    print(f"Leader count:        {profile['leader_count']}")
    print(f"Early count:         {profile['early_count']}")
    print(f"Follower count:      {profile['follower_count']}")
    print(f"Paired count:        {profile['paired_count']}")
    print(f"Noise count:         {profile['noise_count']}")
    print(f"Confidence:          {profile['confidence']}")
    print(f"Dynamic weight:      {profile['dynamic_weight']}")
print("=" * 80)

raw_bet_candidates = [
    g for g in result["scored_candidates"]
    if isinstance(g, dict) and str(g.get("label", "") or "").upper() == "BET"
]

bet_candidates = dedupe_bet_candidates_for_cycle(raw_bet_candidates)

rejected_candidates = [
```

```
    g for g in result["scored_candidates"]
    if isinstance(g, dict) and g["label"] == "PASS"
]

alert_decision_counts = defaultdict(int)
alert_decision_counts["raw_bet_candidates"] = len(raw_bet_candidates)
alert_decision_counts["cycle_deduped_away"] = (
    len(raw_bet_candidates) - len(bet_candidates)
)

new_bet_alerts = []

for g in bet_candidates:
    alert_g = annotate_opposite_side_conflict(g, alerted_bets)
    record_clv_bet(alert_g, clv_tracker, result["now_ts"])

    decision = classify_bet_alert_decision(
        alert_g,
        alerted_bets,
        result["now_ts"],
    )
    alert_decision_counts[decision] += 1

raw_bet_candidates = [
    g for g in result["scored_candidates"]
    if isinstance(g, dict) and str(g.get("label", "") or "").upper() == "BET"
]

bet_candidates = dedupe_bet_candidates_for_cycle(raw_bet_candidates)

rejected_candidates = [
    g for g in result["scored_candidates"]
    if isinstance(g, dict) and g["label"] == "PASS"
]

alert_decision_counts = defaultdict(int)
alert_decision_counts["raw_bet_candidates"] = len(raw_bet_candidates)
alert_decision_counts["cycle_deduped_away"] = (
    len(raw_bet_candidates) - len(bet_candidates)
)

new_bet_alerts = []

for g in bet_candidates:
```

```
    alert_g = annotate_opposite_side_conflict(g, alerted_bets)
    record_clv_bet(alert_g, clv_tracker, result["now_ts"])

    decision = classify_bet_alert_decision(
        alert_g,
        alerted_bets,
        result["now_ts"],
    )
    alert_decision_counts[decision] += 1

    if should_send_bet_alert(alert_g, alerted_bets, result["now_ts"]):
        store_bet_alert(alert_g, alerted_bets, result["now_ts"])
        record_tracked_bet(alert_g, tracked_bets, result["now_ts"])
        record_signal_metrics_row(
            alert_g,
            signal_metrics_history,
            result["now_ts"],
            wallet_profiles,
        )
        send_pushover_bet_alert(alert_g)
        new_bet_alerts.append(alert_g)

cycle_bet_alerts = new_bet_alerts

raw_bet_age_summary = summarize_bet_age_buckets(bet_candidates)
sent_bet_age_summary = summarize_bet_age_buckets(new_bet_alerts)

# --- extract metrics for distribution analysis ---
notional_values = []
ratio_values = []
roi_values = []

for g in bet_candidates:
    if not isinstance(g, dict):
        continue

    notional_values.append(g.get("total_notional", 0))
    ratio_values.append(g.get("size_ratio", 0))
    roi_values.append(g.get("leaderboard_roi", 0))

notional_summary = summarize_numeric_distribution(
    notional_values,
    {
        "<500": lambda x: x < 500,
```

```
            "500-1k": lambda x: x < 1000,
            "1k-5k": lambda x: x < 5000,
            "5k-25k": lambda x: x < 25000,
            "25k-100k": lambda x: x < 100000,
            "100k+": lambda x: True,
        }
    )

    ratio_summary = summarize_numeric_distribution(
        ratio_values,
        {
            "<1": lambda x: x < 1,
            "1-1.5": lambda x: x < 1.5,
            "1.5-2": lambda x: x < 2,
            "2-3": lambda x: x < 3,
            "3-5": lambda x: x < 5,
            "5+": lambda x: True,
        }
    )

    roi_summary = summarize_numeric_distribution(
        roi_values,
        {
            "<0": lambda x: x < 0,
            "0-1%": lambda x: x < 0.01,
            "1-3%": lambda x: x < 0.03,
            "3-5%": lambda x: x < 0.05,
            "5-8%": lambda x: x < 0.08,
            "8%+": lambda x: True,
        }
    )

    print("-" * 80)
    print("BET ALERT DECISION SUMMARY")
    print("=" * 80)
    print(f"Raw BET candidates: {alert_decision_counts['raw_bet_candidates']}")
    print(f"Cycle deduped away: {alert_decision_counts['cycle_deduped_away']}")
    print(f"Sent new bets: {alert_decision_counts['send_new_bet']}")
    print(f"Sent new bets - edge path: {alert_decision_counts['send_new_bet_edge']}")
    print(f"Sent new bets - confirmed follow path:
{alert_decision_counts['send_new_bet_confirmed_follow']}")
    print(f"Sent actionable duplicates: {alert_decision_counts['send_actionable_duplicate']}")
    print(f"Sent possible flips: {alert_decision_counts['send_possible_flip']}")
    print(f"Skipped opposite conflicts: {alert_decision_counts['skip_opposite_conflict']}")
```

```
        print(f"Skipped new bets - weak: {alert_decision_counts['skip_new_bet_weak']}")
        print(f"Skipped duplicate - not actionable:
{alert_decision_counts['skip_duplicate_not_actionable']}")
        print(f"Skipped duplicate - cooldown:
{alert_decision_counts['skip_duplicate_cooldown']}")
        print(f"Skipped duplicate - price worse:
{alert_decision_counts['skip_duplicate_price_worse']}")

        print("-" * 80)
        print("BET AGE SUMMARY - POST-DEDUP BET CANDIDATES")
        print("=" * 80)
        print(f"Count: {len(bet_candidates)}")
        print(f"Avg age (s): {raw_bet_age_summary['avg_age_seconds']}")
        print(f"Median age (s): {raw_bet_age_summary['median_age_seconds']}")
        print(f"00-60s: {raw_bet_age_summary['bucket_counts']['00-60s']}")
        print(f"01-03m: {raw_bet_age_summary['bucket_counts']['01-03m']}")
        print(f"03-05m: {raw_bet_age_summary['bucket_counts']['03-05m']}")
        print(f"05-10m: {raw_bet_age_summary['bucket_counts']['05-10m']}")
        print(f"10-20m: {raw_bet_age_summary['bucket_counts']['10-20m']}")
        print(f"20m+: {raw_bet_age_summary['bucket_counts']['20m+']}")
        print(f"Unknown: {raw_bet_age_summary['bucket_counts']['unknown']}")

        print("-" * 80)
        print("BET AGE SUMMARY - SENT ALERTS")
        print("=" * 80)
        print(f"Count: {len(new_bet_alerts)}")
        print(f"Avg age (s): {sent_bet_age_summary['avg_age_seconds']}")
        print(f"Median age (s): {sent_bet_age_summary['median_age_seconds']}")
        print(f"00-60s: {sent_bet_age_summary['bucket_counts']['00-60s']}")
        print(f"01-03m: {sent_bet_age_summary['bucket_counts']['01-03m']}")
        print(f"03-05m: {sent_bet_age_summary['bucket_counts']['03-05m']}")
        print(f"05-10m: {sent_bet_age_summary['bucket_counts']['05-10m']}")
        print(f"10-20m: {sent_bet_age_summary['bucket_counts']['10-20m']}")
        print(f"20m+: {sent_bet_age_summary['bucket_counts']['20m+']}")
        print(f"Unknown: {sent_bet_age_summary['bucket_counts']['unknown']}")
        print("-" * 80)
        print("NOTIONAL SIZE DISTRIBUTION (POST-DEDUP BET CANDIDATES)")
        print("=" * 80)
        print(f"Avg: {notional_summary['avg']}")
        print(f"Median: {notional_summary['median']}")
        for k, v in notional_summary["bucket_counts"].items():
            print(f"{k}: {v}")

        print("-" * 80)
```

```
        print("SIZE RATIO DISTRIBUTION")
        print("=" * 80)
        print(f"Avg: {ratio_summary['avg']}")
        print(f"Median: {ratio_summary['median']}")
        for k, v in ratio_summary["bucket_counts"].items():
            print(f"{k}: {v}")

        print("-" * 80)
        print("ROI DISTRIBUTION")
        print("=" * 80)
        print(f"Avg: {roi_summary['avg']}")
        print(f"Median: {roi_summary['median']}")
        for k, v in roi_summary["bucket_counts"].items():
            print(f"{k}: {v}")
        clv_summary = update_clv_tracker(
            clv_tracker,
            result["scored_candidates"],
            result["now_ts"],
        )
        tracked_bet_summary = update_tracked_bet_results(
            tracked_bets,
            result["now_ts"],
        )
        save_clv_tracker(clv_tracker)
        save_tracked_bets(tracked_bets)
        save_alerted_bets(alerted_bets)
        save_signal_metrics_history(signal_metrics_history)

        recent_model_signal_metrics = filter_recent_signal_metrics_rows(
            signal_metrics_history,
            MODEL_HISTORY_LOOKBACK_HOURS,
        )
        market_model_recommendations =
build_recommendations(recent_model_signal_metrics)
        save_recommendations_json(market_model_recommendations)

        print("-" * 80)
        print("MARKET MODEL SUMMARY")
        print("=" * 80)
        print(f"Model history lookback hours: {MODEL_HISTORY_LOOKBACK_HOURS}")
        print(f"Recent model signal rows: {len(recent_model_signal_metrics)}")
        print(f"Saved market model recommendations: {len(market_model_recommendations)}")

        top_model_bets = [
```

```
    row for row in market_model_recommendations
    if str(row.get("recommendation", "") or "").upper() == "BET"
]

top_model_leans = [
    row for row in market_model_recommendations
    if str(row.get("recommendation", "") or "").upper() == "LEAN"
]

print(f"Model BET count: {len(top_model_bets)}")
print(f"Model LEAN count: {len(top_model_leans)}")

if top_model_bets:
    top_row = top_model_bets[0]
    print(
        f"Top model BET: {top_row.get('market', 'N/A')} | "
        f"{top_row.get('outcome', 'N/A')} | "
        f"score={top_row.get('model_score', 'N/A')} | "
        f"mins_to_start={top_row.get('minutes_to_start', 'N/A')}"
    )
elif top_model_leans:
    top_row = top_model_leans[0]
    print(
        f"Top model LEAN: {top_row.get('market', 'N/A')} | "
        f"{top_row.get('outcome', 'N/A')} | "
        f"score={top_row.get('model_score', 'N/A')} | "
        f"mins_to_start={top_row.get('minutes_to_start', 'N/A')}"
    )
else:
    print("Top model signal: None")

print("-" * 80)
print("SNAPSHOT CLV SUMMARY")
print("=" * 80)
print(f"Tracked BET alerts: {clv_summary['tracked']}")
print(f"CLV-ready alerts: {clv_summary['ready']}")
print(f"Positive CLV count: {clv_summary['positive']}")
if clv_summary["ready"] > 0:
    positive_rate = round(
        (clv_summary["positive"] / max(clv_summary["ready"], 1)) * 100,
        1,
    )
    print(f"Positive CLV rate: {positive_rate}%")
else:
```

```
    print("Positive CLV rate: N/A")
print(f"Avg snapshot CLV: {clv_summary['avg_snapshot_clv_cents']} cents")
print("-" * 80)
print("TRACKED BET RESULTS SUMMARY")
print("=" * 80)
print(f"Tracked bets: {tracked_bet_summary['tracked']}")
print(f"Resolved bets: {tracked_bet_summary['resolved']}")
print(f"Wins: {tracked_bet_summary['wins']}")
print(f"Losses: {tracked_bet_summary['losses']}")
print(f"Newly resolved this cycle: {tracked_bet_summary['newly_resolved']}")
if tracked_bet_summary["resolved"] > 0:
    tracked_win_rate = round(
        (tracked_bet_summary["wins"] / max(tracked_bet_summary["resolved"], 1)) * 100,
        1,
    )
    print(f"Resolved win rate: {tracked_win_rate}%")
else:
    print("Resolved win rate: N/A")
print("-" * 80)
wallet_result_rows = summarize_tracked_bets_by_wallet(tracked_bets)

print("TRACKED BET RESULTS BY WALLET")
print("=" * 80)
if not wallet_result_rows:
    print("No tracked bets by wallet yet.")
else:
    for row in wallet_result_rows[:15]:
        print("-" * 80)
        print(f"Wallet: {row['wallet']}")
        print(f"Tracked bets: {row['tracked']}")
        print(f"Resolved bets: {row['resolved']}")
        print(f"Wins: {row['wins']}")
        print(f"Losses: {row['losses']}")
        print(f"Win rate: {row.get('win_rate_pct', 'N/A')}")
        print(f"Avg edge at alert: {row['avg_edge_pct_at_alert']}")
        print(f"Avg instant CLV at alert: {row['avg_instant_clv_cents_at_alert']}")
    print("-" * 80)

print_consensus(result["consensus_list"])
print("-" * 80)
print("NEAR-CONSENSUS MISSES")

near_misses = [
    c for c in result["consensus_list"]
```

```
    if c.get("consensus_type") == "near"
]

if not near_misses:
    print("No near-consensus situations.")
else:
    for c in near_misses[:10]:
        print("-" * 80)
        print(f"Market:               {c['market']}")
        print(f"Outcome:              {c['outcome']}")
        print(f"Wallet count (all):    {c['wallet_count_all']}")
        print(f"Wallet count (scored): {c['wallet_count_scored']}")
        print(f"Weighted all:          {c.get('weighted_wallet_score_all', 'N/A')}")
        print(f"Weighted scored:       {c.get('weighted_wallet_score_scored', 'N/A')}")
        print(f"Total size (all):      {round(c['total_size_all'], 2)}")
        print(f"Consensus score:       {c['consensus_score']}")

print("-" * 80)
print("CONSENSUS DIAGNOSTICS")
print("=" * 80)

diagnostics = result.get("consensus_diagnostics", [])

if not diagnostics:
    print("No multi-wallet markets to diagnose.")
else:
    for d in diagnostics[:10]:
        print("-" * 80)
        print(f"Market:              {d['market']}")
        print(f"Outcome:             {d['outcome']}")
        print(f"Wallet count:        {d['wallet_count']}")

        for row in d["rows"]:
            print("  " + "-" * 72)
            print(f"  Wallet:           {row['wallet']}")
            print(f"  Buy count:        {row.get('buy_count', 'N/A')}")
            print(f"  Total size:       {row.get('total_size', 'N/A')}")
            print(f"  Avg trade price:  {row.get('avg_trade_price', 'N/A')}")
            print(f"  Sequence role:    {row.get('sequence_role', 'N/A')}")
            print(f"  Label:            {row.get('label', 'N/A')}")
            print(f"  Reason:           {row.get('reason', 'N/A')}")
            print(f"  Market Movement:  {row.get('market_movement_cents', 'N/A')}")
            print(f"  Since last buy(s): {row.get('seconds_since_last_buy', 'N/A')}")
            print(f"  Size ratio:       {row.get('size_ratio', 'N/A')}")
```

```
        if rejected_candidates:
            print("-" * 80)
            print("TOP REJECTED CANDIDATES")
            for g in rejected_candidates[:10]:
                print("-" * 80)
                print(f"Market:             {g['title']}")
                print(f"Outcome:            {g['outcome']}")
                print(f"Label:              {g['label']}")
                print(f"Reason: {g['reason']}")
                clv_key = make_clv_key(g.get("slug", ""), g.get("outcome", ""), g.get("wallet", ""))
                clv_row = clv_tracker.get(clv_key, {}) if isinstance(clv_tracker, dict) else {}
                snapshot_clv = clv_row.get("snapshot_clv")
                snapshot_clv_ready = clv_row.get("snapshot_clv_ready", False)
                latest_price = clv_row.get("latest_price")
                clv_status = "Not tracked yet"
                if clv_row:
                    if snapshot_clv_ready and snapshot_clv is not None:
                        try:
                            snapshot_clv_cents = round(float(snapshot_clv) * 100, 2)
                            clv_status = f"{snapshot_clv_cents:+.2f} cents"
                        except Exception:
                            clv_status = "Ready but unreadable"
                    else:
                        entry_price_for_clv = clv_row.get("entry_price")
                        try:
                            if entry_price_for_clv is not None and latest_price is not None:
                                live_clv_cents = round((float(latest_price) - float(entry_price_for_clv)) *
100, 2)
                                clv_status = f"Pending ({live_clv_cents:+.2f} cents so far)"
                            else:
                                clv_status = "Pending"
                        except Exception:
                            clv_status = "Pending"
                print(f"Snapshot CLV: {clv_status}")
                market_text = g.get("market")
                if not market_text:
                    market_text = g.get("question")
                if not market_text:
                    market_text = g.get("title")
                if not market_text:
                    market_text = g.get("slug")

    except Exception as e:
```

```
    import traceback
    print(f"[Loop error] {repr(e)}")
    traceback.print_exc()

print("=" * 80)
print("=" * 80)
print("FINAL BET RECOMMENDATIONS")
print("=" * 80)

if cycle_bet_alerts:
    print("\a", end="")
    for g in cycle_bet_alerts[:15]:
        print_signal(g)
else:
    print("No BETs this cycle.")

print("=" * 80)
print(f"Sleeping for {POLL_SECONDS} seconds...")
print("=" * 80)
time.sleep(POLL_SECONDS)
```

market-model.py

```
import json
import os
from collections import defaultdict
from datetime import datetime, timezone


BASE_DIR = os.path.dirname(os.path.abspath(__file__))

if os.path.isdir("/data"):
    DATA_DIR = "/data"
else:
    DATA_DIR = os.path.join(BASE_DIR, "data")

os.makedirs(DATA_DIR, exist_ok=True)

SIGNAL_METRICS_HISTORY_PATH = os.path.join(DATA_DIR, "signal_metrics_history.json")
MARKET_MODEL_OUTPUT_PATH = os.path.join(DATA_DIR, "market_model_output.json")

MODEL_MIN_MINUTES_TO_START = 0
MODEL_MAX_MINUTES_TO_START = 180
MODEL_HISTORY_LOOKBACK_HOURS = 12


def load_signal_metrics_history():
    try:
        with open(SIGNAL_METRICS_HISTORY_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
            if isinstance(data, list):
                return data
    except Exception:
        pass
    return []

def filter_recent_signal_metrics_rows(rows,
lookback_hours=MODEL_HISTORY_LOOKBACK_HOURS):
    if not isinstance(rows, list):
        return []

    now_ts = int(datetime.now(timezone.utc).timestamp())
    cutoff_ts = now_ts - int(lookback_hours * 60 * 60)

    filtered = []

    for row in rows:
```

```
        if not isinstance(row, dict):
            continue

        row_ts = parse_ts(row.get("ts"))
        if row_ts is None:
            continue

        if row_ts >= cutoff_ts:
            filtered.append(row)

    return filtered

def parse_ts(ts_value):
    try:
        return int(ts_value)
    except Exception:
        return None


def parse_float(value, default=0.0):
    try:
        if value is None:
            return default
        return float(value)
    except Exception:
        return default


def parse_int(value, default=0):
    try:
        if value is None:
            return default
        return int(value)
    except Exception:
        return default


def parse_event_start_time(event_start_time):
    if not event_start_time:
        return None

    try:
        return datetime.fromisoformat(str(event_start_time).replace("Z", "+00:00"))
    except Exception:
```

```
        return None


def get_minutes_to_start(row):
    if not isinstance(row, dict):
        return None

    existing = row.get("minutes_to_start")
    if existing is not None:
        try:
            return int(existing)
        except Exception:
            pass

    event_dt = parse_event_start_time(row.get("event_start_time"))
    row_ts = parse_ts(row.get("ts"))

    if event_dt is None or row_ts is None:
        return None

    alert_dt = datetime.fromtimestamp(row_ts, tz=timezone.utc)
    return int((event_dt - alert_dt).total_seconds() / 60)


def get_model_window_bucket(minutes_to_start):
    if minutes_to_start is None:
        return "unknown"

    m = int(minutes_to_start)

    if m < 0:
        return "started"
    if m <= 10:
        return "00-10m"
    if m <= 30:
        return "10-30m"
    if m <= 60:
        return "30-60m"
    if m <= 180:
        return "01-03h"
    if m <= 360:
        return "03-06h"
    if m <= 720:
        return "06-12h"
```

```
    return "12h+"

def is_in_model_window(minutes_to_start):
    if minutes_to_start is None:
        return False

    try:
        m = int(minutes_to_start)
    except Exception:
        return False

    return MODEL_MIN_MINUTES_TO_START <= m <= MODEL_MAX_MINUTES_TO_START

def group_rows_by_market_outcome(rows):
    grouped = defaultdict(list)

    for row in rows:
        if not isinstance(row, dict):
            continue

        slug = str(row.get("slug", "") or "").strip()
        outcome = str(row.get("outcome", "") or "").strip()

        if not slug or not outcome:
            continue

        grouped[(slug, outcome)].append(row)

    for key in grouped:
        grouped[key] = sorted(
            grouped[key],
            key=lambda x: parse_ts(x.get("ts")) or 0,
        )

    return grouped


def build_market_snapshot(rows):
    if not rows:
        return None

    latest = rows[-1]

    wallets = set()
```

```
total_notional = 0.0
total_size = 0.0
size_ratios = []
scores = []
stake_pcts = []
edges = []
market_moves = []
followers = []
consensus_scores = []
leaderboard_rois = []
wallet_counts_by_role = defaultdict(int)

for row in rows:
    wallet = str(row.get("wallet", "") or "").strip().lower()
    if wallet:
        wallets.add(wallet)

    total_notional += parse_float(row.get("total_notional"), 0.0)
    total_size += parse_float(row.get("total_size"), 0.0)

    size_ratio = parse_float(row.get("size_ratio"), None)
    if size_ratio is not None:
        size_ratios.append(size_ratio)

    score = parse_int(row.get("score"), None)
    if score is not None:
        scores.append(score)

    stake_pct = parse_int(row.get("stake_pct"), None)
    if stake_pct is not None:
        stake_pcts.append(stake_pct)

    edge_pct = parse_float(row.get("edge_pct"), None)
    if edge_pct is not None:
        edges.append(edge_pct)

    market_movement_cents = parse_float(row.get("market_movement_cents"), None)
    if market_movement_cents is not None:
        market_moves.append(market_movement_cents)

    follower_count = parse_int(row.get("followers"), None)
    if follower_count is not None:
        followers.append(follower_count)
```

```
        consensus_score = parse_int(row.get("consensus_score"), None)
        if consensus_score is not None:
            consensus_scores.append(consensus_score)

        leaderboard_roi = row.get("leaderboard_roi")
        if leaderboard_roi is not None:
            try:
                leaderboard_rois.append(float(leaderboard_roi))
            except Exception:
                pass

        sequence_role = str(row.get("sequence_role", "") or "").strip().lower()
        if sequence_role:
            wallet_counts_by_role[sequence_role] += 1

    minutes_to_start = get_minutes_to_start(latest)
    time_bucket = get_model_window_bucket(minutes_to_start)

    snapshot = {
        "slug": str(latest.get("slug", "") or "").strip(),
        "market": str(latest.get("market", "") or latest.get("title", "") or "").strip(),
        "outcome": str(latest.get("outcome", "") or "").strip(),
        "market_phase": str(latest.get("market_phase", "") or "").strip(),
        "event_start_time": latest.get("event_start_time"),
        "minutes_to_start": minutes_to_start,
        "time_to_start_bucket": time_bucket,
        "alert_count": len(rows),
        "unique_wallet_count": len(wallets),
        "leader_count": wallet_counts_by_role.get("leader", 0),
        "early_count": wallet_counts_by_role.get("early", 0),
        "follower_count": wallet_counts_by_role.get("follower", 0),
        "total_notional": round(total_notional, 2),
        "total_size": round(total_size, 4),
        "max_size_ratio": round(max(size_ratios), 4) if size_ratios else None,
        "avg_size_ratio": round(sum(size_ratios) / len(size_ratios), 4) if size_ratios else None,
        "max_score": max(scores) if scores else None,
        "avg_score": round(sum(scores) / len(scores), 2) if scores else None,
        "max_stake_pct": max(stake_pcts) if stake_pcts else None,
        "avg_stake_pct": round(sum(stake_pcts) / len(stake_pcts), 2) if stake_pcts else None,
        "avg_edge_pct": round(sum(edges) / len(edges), 4) if edges else None,
        "avg_market_movement_cents": round(sum(market_moves) / len(market_moves), 4) if
market_moves else None,
        "max_followers": max(followers) if followers else 0,
        "max_consensus_score": max(consensus_scores) if consensus_scores else 0,
```

```
        "avg_leaderboard_roi": round(sum(leaderboard_rois) / len(leaderboard_rois), 6) if
leaderboard_rois else None,
        "latest_current_price": latest.get("current_price"),
        "latest_wallet_entry_price": latest.get("wallet_entry_price"),
        "latest_edge_pct": latest.get("edge_pct"),
        "latest_score": latest.get("score"),
        "latest_stake_pct": latest.get("stake_pct"),
        "latest_size_ratio": latest.get("size_ratio"),
        "latest_total_notional": latest.get("total_notional"),
        "latest_followers": latest.get("followers"),
        "latest_consensus_score": latest.get("consensus_score"),
        "latest_sequence_role": latest.get("sequence_role"),
        "latest_ts": latest.get("ts"),
    }

    return snapshot


def score_market_snapshot(snapshot):
    if not isinstance(snapshot, dict):
        return None

    score = 0
    reasons = []

    total_notional = parse_float(snapshot.get("total_notional"), 0.0)
    max_size_ratio = parse_float(snapshot.get("max_size_ratio"), 0.0)
    avg_size_ratio = parse_float(snapshot.get("avg_size_ratio"), 0.0)
    unique_wallet_count = parse_int(snapshot.get("unique_wallet_count"), 0)
    avg_leaderboard_roi = snapshot.get("avg_leaderboard_roi")
    latest_edge_pct = parse_float(snapshot.get("latest_edge_pct"), 0.0)
    minutes_to_start = snapshot.get("minutes_to_start")
    max_followers = parse_int(snapshot.get("max_followers"), 0)
    max_consensus_score = parse_int(snapshot.get("max_consensus_score"), 0)

    if total_notional >= 25000:
        score += 25
        reasons.append("very strong total notional")
    elif total_notional >= 10000:
        score += 18
        reasons.append("strong total notional")
    elif total_notional >= 5000:
        score += 12
        reasons.append("good total notional")
```

```
elif total_notional >= 1000:
    score += 6
    reasons.append("acceptable total notional")

if max_size_ratio >= 5:
    score += 20
    reasons.append("elite size ratio")
elif max_size_ratio >= 3:
    score += 14
    reasons.append("strong size ratio")
elif max_size_ratio >= 1.5:
    score += 8
    reasons.append("solid size ratio")

if unique_wallet_count >= 2 and avg_size_ratio >= 2:
    score += 8
    reasons.append("multiple strong wallets")
elif avg_size_ratio >= 1.2:
    score += 4
    reasons.append("average ratio above baseline")

if unique_wallet_count >= 3:
    score += 15
    reasons.append("multi-wallet support")
elif unique_wallet_count >= 2:
    score += 8
    reasons.append("two-wallet support")

if max_followers >= 2:
    score += 6
    reasons.append("follower support")
elif max_followers >= 1:
    score += 3
    reasons.append("some follower support")

if max_consensus_score >= 70:
    score += 10
    reasons.append("strong consensus score")
elif max_consensus_score >= 50:
    score += 5
    reasons.append("moderate consensus score")

if avg_leaderboard_roi is not None:
    if avg_leaderboard_roi >= 0.05:
```

```
        score += 12
        reasons.append("strong wallet ROI")
    elif avg_leaderboard_roi >= 0.02:
        score += 6
        reasons.append("acceptable wallet ROI")
    elif avg_leaderboard_roi < 0.01:
        score -= 6
        reasons.append("weak wallet ROI")

if latest_edge_pct >= 3:
    score += 10
    reasons.append("strong current edge")
elif latest_edge_pct >= 1:
    score += 5
    reasons.append("positive current edge")
elif latest_edge_pct < -1:
    score -= 8
    reasons.append("edge too far gone")

if minutes_to_start is not None:
    if 0 <= minutes_to_start <= 10:
        score += 12
        reasons.append("very close to start")
    elif 10 < minutes_to_start <= 30:
        score += 10
        reasons.append("close to start")
    elif 30 < minutes_to_start <= 60:
        score += 6
        reasons.append("within one hour")
    elif 60 < minutes_to_start <= 180:
        score -= 4
        reasons.append("too early (1–3h)")
    elif 180 < minutes_to_start <= 360:
        score -= 10
        reasons.append("very early (3–6h)")
    elif minutes_to_start > 360:
        score -= 15
        reasons.append("extremely early (6h+)")
    elif minutes_to_start < 0:
        score -= 20
        reasons.append("market already started")

recommendation = "PASS"
stake_pct = 0
```

```
    if unique_wallet_count >= 3 and score >= 65:
        recommendation = "BET"
        stake_pct = 100
    elif unique_wallet_count >= 2 and score >= 45:
        recommendation = "LEAN"
        stake_pct = 50
    elif unique_wallet_count == 1 and score >= 55:
        recommendation = "LEAN"
        stake_pct = 25

    return {
        "model_score": score,
        "recommendation": recommendation,
        "recommended_stake_pct": stake_pct,
        "reasons": reasons,
    }

def classify_signal_stage(snapshot, model_output):
    if not isinstance(snapshot, dict) or not isinstance(model_output, dict):
        return "discard"

    unique_wallet_count = parse_int(snapshot.get("unique_wallet_count"), 0)
    total_notional = parse_float(snapshot.get("total_notional"), 0.0)
    max_size_ratio = parse_float(snapshot.get("max_size_ratio"), 0.0)
    max_followers = parse_int(snapshot.get("max_followers"), 0)
    max_consensus_score = parse_int(snapshot.get("max_consensus_score"), 0)
    minutes_to_start = parse_int(snapshot.get("minutes_to_start"), -999999)
    model_score = parse_int(model_output.get("model_score"), 0)

    if (
        unique_wallet_count >= 2
        or max_followers >= 1
        or max_consensus_score >= 50
    ):
        if model_score >= 30 and 0 <= minutes_to_start <= MODEL_MAX_MINUTES_TO_START:
            return "confirmed"

    if (
        unique_wallet_count == 1
        and total_notional >= 5000
        and max_size_ratio >= 3
        and model_score >= 30
        and 60 < minutes_to_start <= MODEL_MAX_MINUTES_TO_START
```

```
    ):
        return "early_watch"

    return "discard"

def build_recommendations(rows):
    grouped = group_rows_by_market_outcome(rows)
    recommendations = []

    for _, market_rows in grouped.items():
        snapshot = build_market_snapshot(market_rows)
        if not snapshot:
            continue

        raw_minutes = snapshot.get("minutes_to_start")

        try:
            minutes_to_start = int(raw_minutes)
        except Exception:
            continue

        if minutes_to_start < MODEL_MIN_MINUTES_TO_START:
            continue

        if minutes_to_start > MODEL_MAX_MINUTES_TO_START:
            continue

        snapshot["minutes_to_start"] = minutes_to_start

        model_output = score_market_snapshot(snapshot)
        if not model_output:
            continue

        signal_stage = classify_signal_stage(snapshot, model_output)
        if signal_stage == "discard":
            continue

        row = dict(snapshot)
        row.update(model_output)
        row["signal_stage"] = signal_stage
        recommendations.append(row)

    recommendations.sort(
        key=lambda x: (
```

```
            parse_int(x.get("model_score"), 0),
            parse_float(x.get("total_notional"), 0.0),
            parse_int(x.get("unique_wallet_count"), 0),
        ),
        reverse=True,
    )

    return recommendations


def save_recommendations_json(recommendations):
    early_watch_count = 0
    confirmed_count = 0

    for row in recommendations:
        stage = str(row.get("signal_stage", "") or "").strip().lower()
        if stage == "early_watch":
            early_watch_count += 1
        elif stage == "confirmed":
            confirmed_count += 1

    payload = {
        "generated_at_utc": datetime.now(timezone.utc).isoformat(),
        "recommendation_count": len(recommendations),
        "early_watch_count": early_watch_count,
        "confirmed_count": confirmed_count,
        "recommendations": recommendations,
    }

    try:
        with open(MARKET_MODEL_OUTPUT_PATH, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)
    except Exception as e:
        print(f"[Market model output save error] {repr(e)}")


def print_recommendations(recommendations, limit=25):
    print("=" * 80)
    print("MARKET MODEL RECOMMENDATIONS")
    print("=" * 80)

    if not recommendations:
        print("No recommendations available.")
        return
```

```
    early_watch_count = sum(
        1 for row in recommendations
        if str(row.get("signal_stage", "") or "").lower() == "early_watch"
    )
    confirmed_count = sum(
        1 for row in recommendations
        if str(row.get("signal_stage", "") or "").lower() == "confirmed"
    )

    print(f"Early watch count: {early_watch_count}")
    print(f"Confirmed count: {confirmed_count}")

    for row in recommendations[:limit]:
        print("-" * 80)
        print(f"Market: {row.get('market', 'N/A')}")
        print(f"Outcome: {row.get('outcome', 'N/A')}")
        print(f"Recommendation: {row.get('recommendation', 'N/A')}")
        print(f"Signal stage: {row.get('signal_stage', 'N/A')}")
        print(f"Model score: {row.get('model_score', 'N/A')}")
        print(f"Recommended stake %: {row.get('recommended_stake_pct', 'N/A')}")
        print(f"Minutes to start: {row.get('minutes_to_start', 'N/A')}")
        print(f"Time bucket: {row.get('time_to_start_bucket', 'N/A')}")
        print(f"Unique wallets: {row.get('unique_wallet_count', 'N/A')}")
        print(f"Leader/Early/Follower: {row.get('leader_count', 0)}/{row.get('early_count',
0)}/{row.get('follower_count', 0)}")
        print(f"Total notional: {row.get('total_notional', 'N/A')}")
        print(f"Max size ratio: {row.get('max_size_ratio', 'N/A')}")
        print(f"Avg size ratio: {row.get('avg_size_ratio', 'N/A')}")
        print(f"Avg leaderboard ROI: {row.get('avg_leaderboard_roi', 'N/A')}")
        print(f"Latest edge %: {row.get('latest_edge_pct', 'N/A')}")
        print(f"Max followers: {row.get('max_followers', 'N/A')}")
        print(f"Max consensus score: {row.get('max_consensus_score', 'N/A')}")
        print(f"Reasons: {', '.join(row.get('reasons', []))}")


if __name__ == "__main__":
    print(f"BASE_DIR: {BASE_DIR}")
    print(f"DATA_DIR in use: {DATA_DIR}")
    print(f"SIGNAL_METRICS_HISTORY_PATH: {SIGNAL_METRICS_HISTORY_PATH}")
    print(f"MARKET_MODEL_OUTPUT_PATH: {MARKET_MODEL_OUTPUT_PATH}")
    print(
        f"MODEL WINDOW: {MODEL_MIN_MINUTES_TO_START} to "
        f"{MODEL_MAX_MINUTES_TO_START} minutes before start"
```

```
    )
    print(f"MODEL HISTORY LOOKBACK HOURS:
{MODEL_HISTORY_LOOKBACK_HOURS}")

    signal_metrics_history = load_signal_metrics_history()
    recent_signal_metrics_history = filter_recent_signal_metrics_rows(
        signal_metrics_history,
        MODEL_HISTORY_LOOKBACK_HOURS,
    )

    print(f"Loaded signal metrics rows: {len(signal_metrics_history)}")
    print(f"Recent signal metrics rows: {len(recent_signal_metrics_history)}")

    recommendations = build_recommendations(recent_signal_metrics_history)
    save_recommendations_json(recommendations)
    print_recommendations(recommendations)
```